

Ενότητα 1. Δομές Δεδομένων και Αλγόριθμοι

1.1 Στοίβα

Δύο τριτοετείς φοιτητές αποφάσισαν τον Δεκαπενταύγουστο να πάρουν το αυτοκίνητό τους και να περάσουν ένα triήμερο στην Αίγινα. Θεωρούσαν ότι το οχηματαγωγό πλοίο χωρητικότητας 30 αυτοκινήτων, που εκτελεί το δρομολόγιο ΠΕΙΡΑΙΑΣ – ΑΙΓΙΝΑ ήταν αρκετά μεγάλο και δεν ήταν απαραίτητη η εκ των προτέρων κράτηση θέσης αυτοκινήτου. Όταν όμως έφθασαν στο λιμάνι, πληροφορήθηκαν, ότι το πλοίο ήταν πλήρες από οχήματα και μόνο σε περίπτωση ακύρωσης θα ελευθερώνονταν θέση για το αυτοκίνητό τους. Αυτό όμως θα το μάθαιναν λίγο πριν τον απόπλου. Τελικά, για καλή τους τύχη, έγινε μία ακύρωση και επιβιβάστηκαν τελευταίοι στο πλοίο μαζί με το αυτοκίνητό τους. Τότε, συνειδητοποίησαν ότι, το πλοίο είχε μία πόρτα, την ίδια για είσοδο και έξοδο των οχημάτων και θα αποβιβάζονταν πρώτοι αποφεύγοντας την ταλαιπωρία της αναμονής.

Παρατήρησαν, μάλιστα, ότι τα οχήματα που μπήκαν πρώτα ήταν σταθμευμένα στο βάθος του πλοίου **σε μία σειρά** το ένα μετά το άλλο και τα τελευταία συμπλήρωναν **αυτή τη σειρά** μέχρι και την πόρτα και μέχρι το πλήθος των 30 αυτοκινήτων.

Δηλαδή, η κατασκευή του «γκαράζ» του πλοίου και ο τρόπος στάθμευσης των οχημάτων προσομοιάζε στη δομή της «Στοίβας».



Στοίβα (stack), ονομάζεται μια δομή δεδομένων το σύνολο των στοιχείων της οποίας είναι διατεταγμένο με τέτοιο τρόπο, ώστε τα στοιχεία που βρίσκονται στην κορυφή της στοίβας λαμβάνονται πρώτα, ενώ αυτά που βρίσκονται στο βάθος της στοίβας λαμβάνονται τελευταία.

Η παραπάνω μέθοδος ονομάζεται **Τελευταίο Μέσα, Πρώτο Έξω ή LIFO** (=Last In First Out).

Μπορούμε να φανταστούμε την τοποθέτηση των στοιχείων μιας στοίβας σε κατακόρυφη σειρά. Χαρακτηριστικό παράδειγμα είναι μια στοίβα από πιάτα. Παίρνουμε προς χρήση το πιάτο που τοποθετήσαμε τελευταίο.



Οι **κύριες λειτουργίες** σε μια στοίβα είναι δύο:

1. Η **ώθηση** (push) στοιχείου στην κορυφή της στοίβας.
Στη διαδικασία της ώθησης ελέγχουμε αν η στοίβα είναι γεμάτη.
Στην περίπτωση που προσπαθήσουμε να «προσθέσουμε» ένα στοιχείο σε μια ήδη γεμάτη στοίβα, έχουμε **υπερχειλίσση** (overflow) της στοίβας.
2. Η **απώθηση** (pop) στοιχείου από τη στοίβα.
Στη διαδικασία της απώθησης ελέγχουμε αν υπάρχει ένα τουλάχιστον στοιχείο στη στοίβα.
Στην περίπτωση που προσπαθήσουμε να «αφαιρέσουμε» ένα στοιχείο από μία κενή στοίβα, έχουμε **υποχειλίσση** (underflow) της στοίβας.

Υλοποίηση στοίβας με χρήση μονοδιάστατου πίνακα

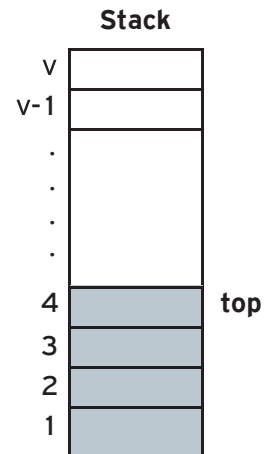
- Χρησιμοποιούμε μια βοηθητική μεταβλητή (top), που δείχνει το στοιχείο που τοποθετήθηκε τελευταίο στην κορυφή της στοίβας.
- Η **ώθηση** ενός νέου στοιχείου στη στοίβα (εισαγωγή στοιχείου στον πίνακα) γίνεται πάντα στην κορυφή της. Συγκεκριμένα, η μεταβλητή top αυξάνεται κατά ένα:

$$top \leftarrow top + 1$$

και στη συνέχεια γίνεται η ώθηση του στοιχείου.

- Η **απώθηση** ενός στοιχείου από τη στοίβα (εξαγωγή από τον πίνακα) γίνεται πάντα από την κορυφή της στοίβας. Συγκεκριμένα, εξάγεται το στοιχείο που δείχνει η μεταβλητή top και στη συνέχεια η μεταβλητή top μειώνεται κατά ένα:

$$top \leftarrow top - 1$$



Εικόνα 1. 1.
Υλοποίηση Στοίβας



Σε μια κενή στοίβα/πίνακα θεωρούμε ότι η αρχική τιμή της μεταβλητής top είναι μηδέν ($top \leftarrow 0$). Η μεταβλητή top είναι η μεταβλητή που δείχνει τη θέση που τοποθετήθηκε το τελευταίο στοιχείο στη στοίβα/πίνακα (δηλ. δείχνει την κορυφή της στοίβας).

Κατά την **ώθηση** ενός στοιχείου στη στοίβα (εισαγωγή ενός στοιχείου στον πίνακα), πρώτα αυξάνεται η τιμή της μεταβλητής top κατά ένα, δηλ. $top \leftarrow top + 1$, και στη συνέχεια γίνεται η ώθηση του στοιχείου στην κορυφή της στοίβας.

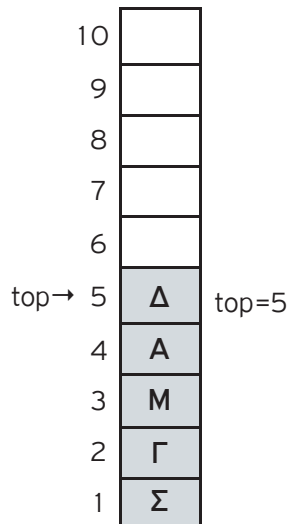
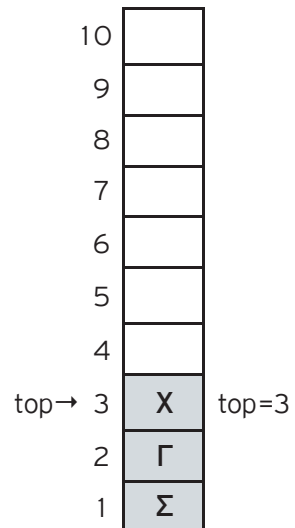
Κατά την **απώθηση** ενός στοιχείου από τη στοίβα (εξαγωγή στοιχείου από τον πίνακα) μειώνεται η τιμή της μεταβλητής top κατά ένα, δηλ. $top \leftarrow top - 1$. Στην απώθηση δε διαγράφεται το στοιχείο, στην πραγματικότητα δε γίνεται καμία παρέμβαση στα περιεχόμενα του πίνακα. Απλώς ο δείκτης κορυφή δείχνει στην προηγούμενη θέση.

1.1.1 Παραδείγματα υλοποίησης στοίβας με χρήση μονοδιάστατου πίνακα

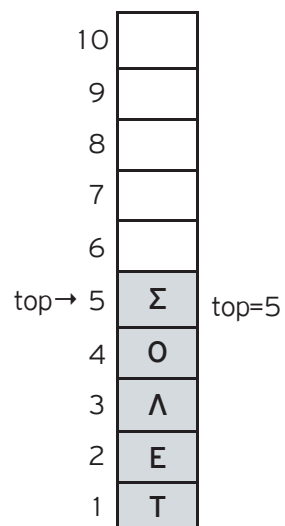
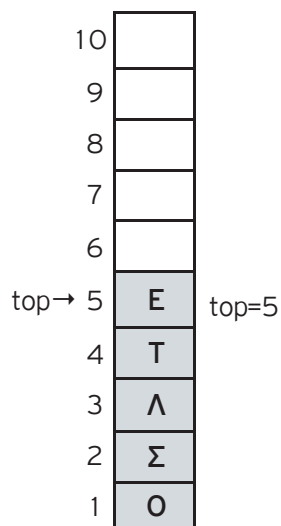


Παράδειγμα 1 – Ώθηση & Απώθηση στοιχείου σε στοίβα

- Σε μια στοίβα 10 θέσεων έχουν τοποθετηθεί διαδοχικά τα στοιχεία: Σ, Γ, Μ, Α, Δ στην 1η, 2η, 3η, 4η και 5η θέση αντίστοιχα.
 - Να προσδιορίσετε την τιμή του δείκτη top και να σχεδιάσετε την παραπάνω στοίβα.
 - Αν εφαρμόσετε τις παρακάτω λειτουργίες: **Απώθηση, Απώθηση, Απώθηση, Ώθηση Χ, Ώθηση Δ** και **Απώθηση**, ποια είναι η νέα τιμή της top και ποια η τελική μορφή της στοίβας;
- Σε μια άδεια στοίβα 10 θέσεων ωθούνται τα στοιχεία Ο, Σ, Λ, Τ, Ε. Με ποιον τρόπο πρέπει να γίνει η ώθηση και η απώθηση των στοιχείων, ώστε να έχουμε ως έξοδο τα στοιχεία Τ, Ε, Λ, Ο, Σ, με το στοιχείο Σ να βρίσκεται στην κορυφή της στοίβας;

Απάντηση**1) i)****ii)** Η νέα τιμή της top είναι 3 και η στοίβα γίνεται:**2)** Η αρχική μορφή της στοίβας είναι:

Εκτελώντας τις λειτουργίες : **Απώθηση, Απώθηση, Απώθηση, Απώθηση, Απώθηση, Ώθηση Τ, Ώθηση Ε, Ώθηση Λ, Ώθηση Ο, Ώθηση Σ**, τότε η τελική μορφή της στοίβας γίνεται:





Παράδειγμα 2 – Ώθηση στοιχείου στην κορυφή της στοίβας με χρήση μονοδιάστατου πίνακα

Να αναπτύξετε τμήμα προγράμματος σε ΓΛΩΣΣΑ, που πραγματοποιεί ώθηση στοιχείου στην κορυφή της στοίβας με χρήση μονοδιάστατου πίνακα A, 10 θέσεων.

Απάντηση

Θεωρούμε ότι η στοίβα δεν είναι κενή και η μεταβλητή top έχει μια έγκυρη τιμή.



Κώδικας σε ΓΛΩΣΣΑ [1.1]

```

1  ΓΡΑΨΕ 'Δώσε στοιχείο για να εισαχθεί στη στοίβα A:'
2  ΔΙΑΒΑΣΕ στοιχείο
3  ΑΝ top<10 ΤΟΤΕ
4      top<- top+1
5      A[top]<- στοιχείο
6  ΑΛΛΙΩΣ
7      ΓΡΑΨΕ 'Υπερχείλιση στοίβας'
8  ΤΕΛΟΣ_ΑΝ

```



Παράδειγμα 3 – Απώθηση στοιχείου από στοίβα με χρήση μονοδιάστατου πίνακα

Να αναπτύξετε τμήμα προγράμματος σε ΓΛΩΣΣΑ που πραγματοποιεί την απώθηση στοιχείου από στοίβα με χρήση ενός μονοδιάστατου πίνακα A, 10 θέσεων

Απάντηση

Θεωρούμε ότι η στοίβα δεν είναι κενή και η μεταβλητή top έχει μια έγκυρη τιμή.



Κώδικας σε ΓΛΩΣΣΑ [1.2]

```

1  ΑΝ top>=1 ΤΟΤΕ
2      ΓΡΑΨΕ A[top]
3      top<- top-1
4  ΑΛΛΙΩΣ
5      ΓΡΑΨΕ 'Υποχείλιση στοίβας'
6  ΤΕΛΟΣ_ΑΝ

```



Παράδειγμα 4 – Επιβίβαση & Αποβίβαση αυτοκινήτων σε πλοίο

Ένα οχηματαγωγό πλοίο, χωρητικότητας 250 αυτοκινήτων, τα οποία δύνανται να τοποθετηθούν αποκλειστικά σε μία σειρά, εκτελεί το δρομολόγιο ΠΕΙΡΑΙΑΣ – ΑΙΓΙΝΑ. Στο λιμάνι του Πειραιά προσέρχονται τα οχήματα για αναχώρηση. Τα οχήματα που επιβιβάζονται πρώτα είναι αυτά που θα αποβιβαστούν τελευταία.

Να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ το οποίο:

1. Να υλοποιεί μενού με τις επιλογές:
 1. Επιβίβαση 2. Αποβίβαση 3. Έξοδος
2. Στην περίπτωση που επιλεγεί η **Επιβίβαση**, να ζητείται εισαγωγή του αριθμού κυκλοφορίας καθενός από τα οχήματα που προσέρχονται και ο αριθμός κυκλοφορίας του να καταχωρείται στη στοίβα ΟΧΗΜΑΤΑ. Κάθε φορά που επιβιβάζεται ένα όχημα να τυπώνεται το ερώτημα «**Υπάρχει όχημα για επιβίβαση; (N/O)**». Αν ο χρήστης απαντήσει N (=ΝΑΙ), τότε να επαναλαμβάνεται η διαδικασία επιβίβασης, ενώ αν απαντήσει O (=ΟΧΙ), τότε να σταματά η διαδικασία επιβίβασης και το πρόγραμμα να επιστρέφει στο μενού Επιλογής.
3. Στην περίπτωση που επιλεγεί η **Αποβίβαση**, να τυπώνει τον αριθμό κυκλοφορίας όλων των οχημάτων με τη σειρά που αποβιβάζονται από το πλοίο, καθώς και το πλήθος των οχημάτων που αποβιβάστηκαν στο λιμάνι της ΑΙΓΙΝΑΣ.
4. Στο τέλος να τυπώνει το πλήθος των οχημάτων που επιβιβάστηκαν στο λιμάνι του ΠΕΙΡΑΙΑ.

Απάντηση



Παρακάτω υλοποιείται το πρόγραμμα του παραδείγματος επιβίβασης/αποβίβασης σε οχηματαγωγό πλοίο, όπου η «πρόρτα» για την είσοδο και την έξοδο των οχημάτων στο πλοίο, είναι ίδια.

Ανάλυση της λύσης

Σύμφωνα με την εκφώνηση της άσκησης πρέπει:

- ο Να υλοποιηθεί μια δομή δεδομένων LIFO (στοίβα). Η υλοποίηση της στοίβας θα πραγματοποιηθεί με μονοδιάστατο πίνακα, όπου οι θέσεις του πίνακα θα είναι τόσες όσες και το μέγιστο πλήθος επιβιβαζόμενων οχημάτων (250). Επομένως για την υλοποίηση της στοίβας θα χρησιμοποιηθεί ένας μονοδιάστατος πίνακας **n** 250 θέσεων, π[250].
- ο Θα πρέπει να δηλωθεί η βοηθητική μεταβλητή (δείκτης) «**τοπ**», που δείχνει το στοιχείο που έχει τοποθετηθεί τελευταίο στη στοίβα. Η αρχική τιμή της μεταβλητής «**τοπ**» πρέπει να είναι 0 για να δείχνει στη «θέση» 0 του πίνακα, επισημαίνοντας κατά αυτόν τον τρόπο ότι το πλοίο/στοίβα είναι κενό. Η μεταβλητή «**τοπ**» αυξάνεται κάθε φορά κατά 1 ($\text{τοπ} \leftarrow \text{τοπ} + 1$), και δείχνει την κενή θέση, την οποία καταλαμβάνει κάθε νέο επιβιβαζόμενο όχημα στο πλοίο.
- ο Να δηλωθούν οι μεταβλητές: **επ1** (επιλογή μενού), **πλ1** (μετρητής επιβιβαζόμενων οχημάτων, με αρχική τιμή 0), **πλ2** (μετρητής αποβιβαζόμενων οχημάτων, με αρχική τιμή 0), **επ2** (μεταβλητή δήλωσης οχήματος για επιβίβαση), **αρ** (αριθμός κυκλοφορίας οχήματος, μέσω του οποίου καταχωρίζεται-δηλώνεται η επιβίβασή του), π[250].

- Ο Να εμφανίζεται ένα μενού με τις επιλογές: **1.** «Επιβίβαση», **2.** «Αποβίβαση», **3.** «Εξοδος» (χρήση δομής επανάληψης «**ΜΕΧΡΙΣ_ΟΤΟΥ...**», μεταβλητή επιλογών «**επ1**»)
 - Ο Στην περίπτωση της **επιβίβασης** («**ΑΝ επ1 = 1 ΤΟΤΕ**»), θα πρέπει το πρόγραμμα να ελέγχει αν υπάρχει όχημα για επιβίβαση (μεταβλητή «**επ2**»). Αν υπάρχει όχημα για επιβίβαση, θα πρέπει παράλληλα να γίνεται έλεγχος αν υπάρχει κενή θέση στο πλοίο («**τοπ < 250**»). Αν υπάρχει κενή θέση, αυξάνεται κατά ένα η τιμή της μεταβλητής «**τοπ**» («**τοπ <- τοπ + 1**») και στη συνέχεια γίνεται καταχώριση του αριθμού κυκλοφορίας του οχήματος σε αυτή την κενή θέση της στοίβας/πίνακα («**π[τοπ] <- αρ**»). Αμέσως μετά την τυχόν καταχώριση του αυτοκινήτου, η τιμή του μετρητή καταχώρισης επιβιβαζόμενων οχημάτων αυξάνεται κατά 1 («**πλ1 <- πλ1 + 1**»). Σε αντίθετη περίπτωση (γεμάτο πλοίο/υπερχείλιση στοίβας), να μη γίνεται καμιά καταχώριση και να εμφανίζεται το μήνυμα («**ΓΡΑΨΕ 'Το πλοίο γέμισε και δε χωρά άλλα αυτοκίνητα'**»).
 - Ο Στην περίπτωση της **αποβίβασης** («**ΑΛΛΙΩΣ_ΑΝ επ1=2 ΤΟΤΕ**») θα πρέπει να ελέγχεται αν υπάρχει αυτοκίνητο για αποβίβαση, μέσω της μεταβλητής «**τοπ**» («**ΟΣΟ τοπ >= 1 ΕΠΑΝΑΛΑΒΕ**»). Αν υπάρχει όχημα προς αποβίβαση, τυπώνεται ο αριθμός κυκλοφορίας του αποβιβαζόμενου οχήματος με κατάλληλο μήνυμα («**ΓΡΑΨΕ 'Αποβιβάζεται το όχημα: ', π[τοπ]**»). Κατόπιν μειώνεται η τιμή του δείκτη «**τοπ**» κατά 1 («**τοπ <- τοπ - 1**»), για να δείχνει στην προηγούμενη θέση της στοίβας/πίνακα, θέση που περιέχει τον αριθμό κυκλοφορίας του επόμενου οχήματος προς αποβίβαση. Παράλληλα, αυξάνουμε την τιμή του μετρητή καταχώρισης αποβιβαζόμενων οχημάτων («**πλ2 <- πλ2 + 1**»). Σε αντίθετη περίπτωση, αν ο δείκτης «**τοπ**» από την αρχή δείχνει 0 (υποχείλιση, δεν υπάρχουν αυτοκίνητα για αποβίβαση ή έχουν αποβιρασθεί όλα τα αυτοκίνητα), θα εμφανίζεται κατάλληλο μήνυμα με το σύνολο των αποβιρασθέντων οχημάτων («**ΓΡΑΨΕ 'Οχήματα που αποβιβάστηκαν στην ΑΙΓΙΝΑ: ', πλ2**»).
- Να σημειωθεί ότι αν δεν αποβιρασθούν οχήματα ή για κάθε νέα επιλογή επιβίβασης ή αποβίβασης (νέα εκτέλεση του μενού), η μεταβλητή πλ2 έχει την αρχική της τιμή, που είναι 0.
- Ο Τέλος, με την εκτύπωση του πλήθους επιβιρασθέντων οχημάτων στον Πειραιά («**ΓΡΑΨΕ 'Οχήματα που επιβιβάστηκαν στον ΠΕΙΡΑΙΑ: ', πλ1**»), γίνεται και τυπικός έλεγχος μεταξύ των οχημάτων που έχουν επιβιρασθεί στο πλοίο κι έχουν αποβιρασθεί από αυτό.

Ακολουθεί η υλοποίηση του προγράμματος σε ΓΛΩΣΣΑ.



Κώδικας σε ΓΛΩΣΣΑ [1.3]

ΠΡΟΓΡΑΜΜΑ πλοίο

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ: τοπ, επ1, πλ1, πλ2

ΧΑΡΑΚΤΗΡΕΣ: επ2, αρ, π[250]

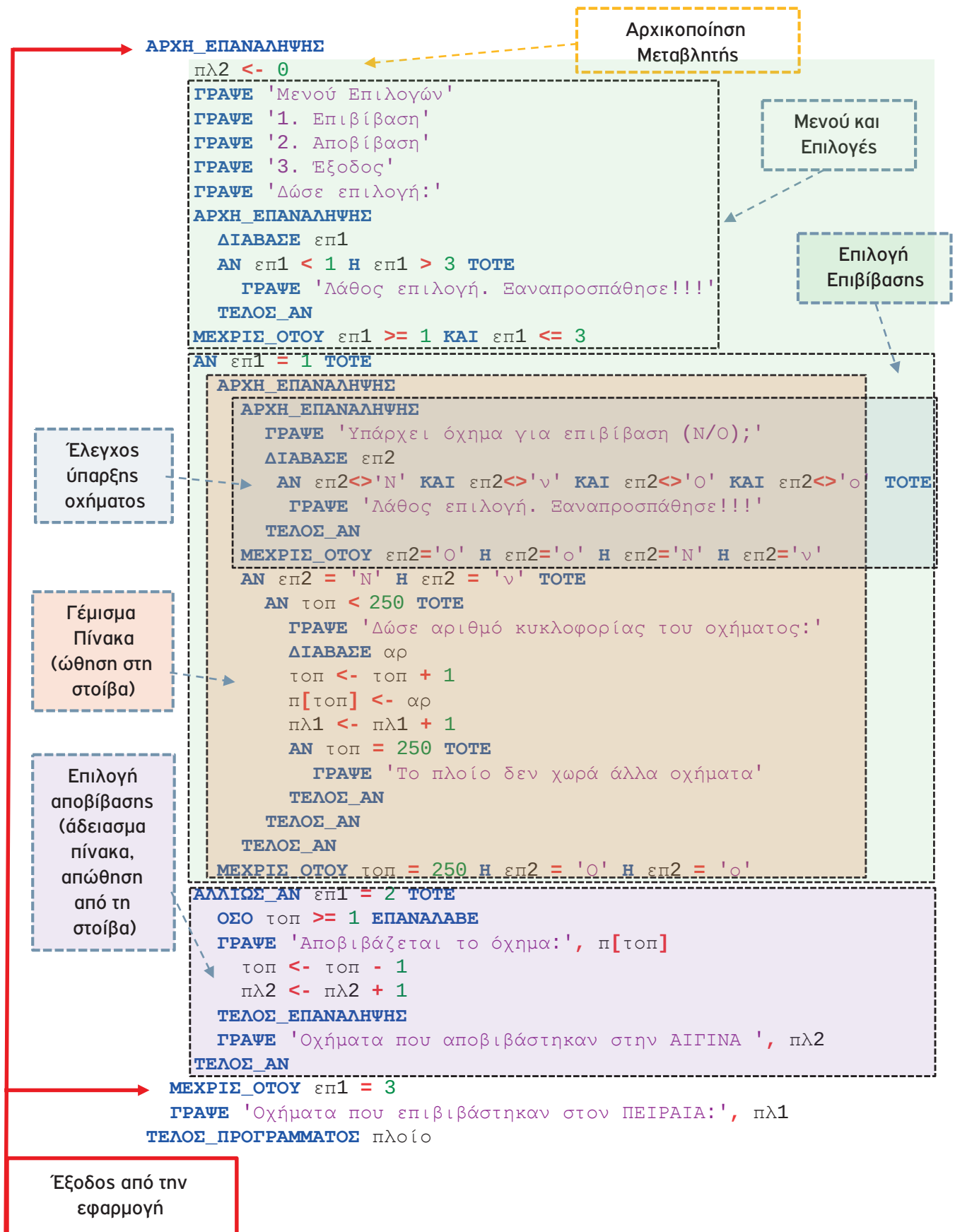
ΑΡΧΗ

τοπ <- 0

πλ1 <- 0

Δήλωση Μεταβλητών

Αρχικοποίηση
Μεταβλητών



```

1  ΠΡΟΓΡΑΜΜΑ πλοίο
2  ΜΕΤΑΒΛΗΤΕΣ
3      ΑΚΕΡΑΙΕΣ: τοπ, επ1, πλ1, πλ2
4      ΧΑΡΑΚΤΗΡΕΣ: επ2, αρ, π[250]
5  ΑΡΧΗ
6      τοπ <- 0
7      πλ1 <- 0
8  ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
9      πλ2 <- 0
10     ΓΡΑΨΕ 'Μενού Επιλογών'
11     ΓΡΑΨΕ '1. Επιβίβαση'
12     ΓΡΑΨΕ '2. Αποβίβαση'
13     ΓΡΑΨΕ '3. Έξοδος'
14     ΓΡΑΨΕ 'Δώσε επιλογή:'
15  ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
16     ΔΙΑΒΑΣΕ επ1
17     ΑΝ επ1 < 1 Η επ1 > 3 ΤΟΤΕ
18         ΓΡΑΨΕ 'Λάθος επιλογή. Ξαναπροσπάθησε!!!'
19     ΤΕΛΟΣ_ΑΝ
20  ΜΕΧΡΙΣ_ΟΤΟΥ επ1 >= 1 ΚΑΙ επ1 <= 3
21  ΑΝ επ1 = 1 ΤΟΤΕ
22     ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
23     ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
24         ΓΡΑΨΕ 'Υπάρχει όχημα για επιβίβαση (N/O);'
25         ΔΙΑΒΑΣΕ επ2
26         ΑΝ επ2<>'N' ΚΑΙ επ2<>'ν' ΚΑΙ επ2<>'Ο' ΚΑΙ επ2<>'ο' ΤΟΤΕ
27             ΓΡΑΨΕ 'Λάθος επιλογή. Ξαναπροσπάθησε!!!'
28         ΤΕΛΟΣ_ΑΝ
29     ΜΕΧΡΙΣ_ΟΤΟΥ επ2='Ο' Η επ2='ο' Η επ2='N' Η επ2='ν'
30     ΑΝ επ2 = 'N' Η επ2 = 'ν' ΤΟΤΕ
31         ΑΝ τοπ < 250 ΤΟΤΕ
32             ΓΡΑΨΕ 'Δώσε αριθμό κυκλοφορίας του οχήματος:'
33             ΔΙΑΒΑΣΕ αρ
34             τοπ <- τοπ + 1
35             π[τοπ] <- αρ
36             πλ1 <- πλ1 + 1
37         ΑΝ τοπ = 250 ΤΟΤΕ
38             ΓΡΑΨΕ 'Το πλοίο δεν χωρά άλλα οχήματα'
39         ΤΕΛΟΣ_ΑΝ
40     ΤΕΛΟΣ_ΑΝ

```



```

41         ΤΕΛΟΣ_ΑΝ
42     ΜΕΧΡΙΣ_ΟΤΟΥ τοπ = 250 Η επ2 = 'Ο' Η επ2 = 'ο'
43     ΑΛΛΙΩΣ_ΑΝ επ1 = 2 ΤΟΤΕ
44         ΟΣΟ τοπ >= 1 ΕΠΑΝΑΛΑΒΕ
45             ΓΡΑΨΕ 'Αποβιβάζεται το όχημα:', π[τοπ]
46             τοπ <- τοπ - 1
47             πλ2 <- πλ2 + 1
48         ΤΕΛΟΣ_ΕΠΑΝΑΛΗΨΗΣ
49         ΓΡΑΨΕ 'Οχήματα που αποβιβάστηκαν στην ΑΙΓΙΝΑ ', πλ2
50     ΤΕΛΟΣ_ΑΝ
51     ΜΕΧΡΙΣ_ΟΤΟΥ επ1 = 3
52     ΓΡΑΨΕ 'Οχήματα που επιβιβάστηκαν στον ΠΕΙΡΑΙΑ:', πλ1
53 ΤΕΛΟΣ_ΠΡΟΓΡΑΜΜΑΤΟΣ πλοίο

```

1.1.2 Ερωτήσεις - Ασκήσεις

Ε. 1: Δίνεται η επόμενη ακολουθία αριθμών: 4, 8, 2, 5, 9, 13.

- Ποια λειτουργία θα χρησιμοποιηθεί για την τοποθέτηση των αριθμών σε στοίβα;
- Σχεδιάστε τη στοίβα μετά την τοποθέτηση των αριθμών.
- Ποια λειτουργία θα χρησιμοποιηθεί για την έξοδο των αριθμών από τη στοίβα;
- Πόσες φορές θα πρέπει να εκτελεστεί η προηγούμενη λειτουργία στη στοίβα για να εξαχθεί ο αριθμός 5;

Ε. 2: Σε μια στοίβα έχουν τοποθετηθεί κατά σειρά οι αριθμοί : 24, 7, 11, 13, 65, 39, 5.

- Να σχεδιάσετε την παραπάνω δομή.
- Ποια θα είναι η τιμή του δείκτη της παραπάνω στοίβας;
- Αν θέλετε να τοποθετήσετε τον αριθμό 25 στην στοίβα, ποια λειτουργία θα χρησιμοποιήσετε;
- Ποια θα είναι η τιμή του δείκτη μετά την λειτουργία αυτή;
- Αν θέλετε να εξάγετε τον αριθμό 65 από τη στοίβα, ποια λειτουργία θα χρησιμοποιήσετε;
- Ποια θα είναι η τιμή του δείκτη μετά τη λειτουργία αυτή;

Ε. 3: Χαρακτηρίστε τις παρακάτω προτάσεις ως Σωστές ή Λάθος. Στην περίπτωση που πιστεύετε ότι είναι λανθασμένες δικαιολογήστε την επιλογή σας και σκεφτείτε ποια θα μπορούσε να είναι η αντίστοιχη σωστή πρόταση.

α/α	Προτάσεις	Σ	Λ
1	Για την υλοποίηση μιας στοίβας μπορεί να χρησιμοποιηθεί ένας πίνακας.	☐	☐
2	Στη στοίβα το στοιχείο που μπαίνει πρώτο βγαίνει πρώτο.	☐	☐
3	Στην υλοποίηση της στοίβας χρειάζονται δύο μεταβλητές-δείκτες για την υλοποίηση των δύο βασικών λειτουργιών που εκτελούνται σε αυτή.	☐	☐
4	Η λειτουργία της ώθησης μπορεί να εκτελεστεί και σε μια άδεια στοίβα.	☐	☐
5	Η λειτουργία της ώθησης μπορεί να εκτελεστεί και σε μια γεμάτη στοίβα.	☐	☐
6	Η ώθηση στοιχείου γίνεται στην κορυφή της στοίβας.	☐	☐
7	Στη δομή της στοίβας απαιτούνται δύο δείκτες, ο εμπρός και ο πίσω.	☐	☐
8	Υπερχείλιση έχουμε όταν εισάγουμε ένα στοιχείο σε μια ήδη γεμάτη στοίβα.	☐	☐
9	Η μέθοδος LIFO περιγράφει τη διαδικασία εκείνη κατά την οποία το στοιχείο που τοποθετείται τελευταίο εξάγεται πρώτο	☐	☐
10	Κάθε στοιχείο που εισάγεται πρώτο σε μια στοίβα είναι αυτό που εξάγεται πρώτο.	☐	☐

1.2 Ουρά

Κατά την ημέρα αναχώρησης τους από την Αίγινα, οι δύο συμφοιτητές αποφάσισαν σκόπιμα να καθυστερήσουν, προκειμένου να μπουν τελευταίοι (ώστε να βγουν πρώτοι και χωρίς ταλαιπωρία στον Πειραιά) και ταυτόχρονα, να απολαύσουν για περισσότερο χρόνο τις τελευταίες στιγμές των διακοπών τους στο νησί.

Δυστυχώς, όμως, ατύχησαν! Το πλοίο της επιστροφής τους ήταν διαφορετικό. Είχε μία είσοδο στο πίσω μέρος προς επιβίβαση των οχημάτων και μία έξοδο στο μπροστινό μέρος προς αποβίβαση. Έτσι, το αυτοκίνητο που έμπαινε πρώτο, προχωρούσε ευθεία προς την άλλη μεριά του πλοίου και έπαιρνε θέση προς αποβίβαση προς την πόρτα εξόδου. Κάθε αυτοκίνητο που έμπαινε στο πλοίο πάκκαρε υποχρεωτικά πίσω από τα άλλα, που είχαν μπει νωρίτερα, στην ίδια σειρά. Δηλαδή, το αυτοκίνητο που επιβιβαζόταν πρώτο στην Αίγινα θα αποβιβαζόταν πρώτο και στον Πειραιά. Με τον τρόπο αυτό, η επιβίβαση και η αποβίβαση των οχημάτων γινόταν πιο γρήγορα. Η κατασκευή του «γκαράζ» του πλοίου και ο τρόπος στάθμευσης των οχημάτων (τρόπος οργάνωσης των δεδομένων) προσομοίαζε στη δομή της «Ουράς». Δηλαδή, τα δεδομένα που μπαίνουν πρώτα σε μια ουρά είναι αυτά που βγαίνουν και πρώτα.



Ουρά (Queue), ονομάζεται μια δομή δεδομένων το σύνολο των στοιχείων της οποίας είναι διατεταγμένο με τέτοιο τρόπο, ώστε τα στοιχεία που τοποθετήθηκαν πρώτα στην ουρά να λαμβάνονται επίσης πρώτα.

Η παραπάνω μέθοδος ονομάζεται **Πρώτο Μέσα, Πρώτο Έξω ή FIFO** (=First In First Out).

Μπορούμε να φανταστούμε την τοποθέτηση των στοιχείων μιας ουράς σε οριζόντια σειρά. Ο πελάτης μιας τράπεζας που μπαίνει πρώτος σε μια ουρά για να εξυπηρετηθεί, είναι αυτός που εξυπηρετείται και πρώτος, με την έξοδό του από την ουρά αναμονής.

Άλλα παραδείγματα είναι η ουρά στα λεωφορεία ή η ουρά στα ταμεία, όπου ο πρώτος που στέκεται στην ουρά εξυπηρετείται και πρώτος.

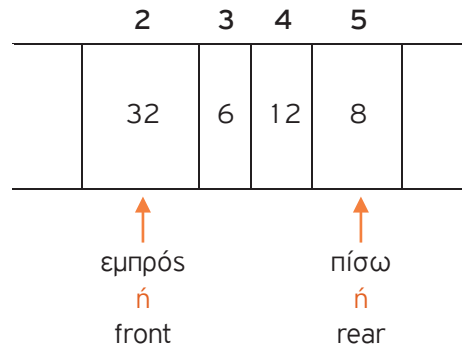


Οι **κύριες λειτουργίες** που εκτελούνται σε μια ουρά είναι δύο:

1. Η **εισαγωγή** (enqueue) στοιχείου στο πίσω άκρο της ουράς.
2. Η **εξαγωγή** (dequeue) στοιχείου από το εμπρός άκρο της ουράς.

Υλοποίηση ουράς με χρήση μονοδιάστατου πίνακα

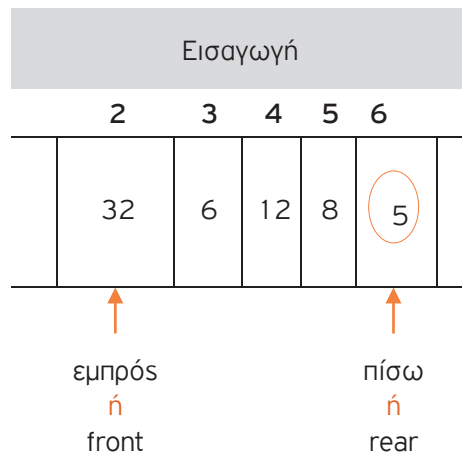
- Χρησιμοποιούμε δύο μεταβλητές, την *front* (ή εμπρός) που δείχνει τη θέση του 1^{ου} στοιχείου της ουράς και την *rear* (ή πίσω) που δείχνει τη θέση του τελευταίου στοιχείου. Ως αρχικές τιμές των μεταβλητών *rear* και *front* θεωρούμε το μηδέν.



- Η **εισαγωγή** ενός νέου στοιχείου γίνεται από το πίσω άκρο της ουράς και η τιμή της μεταβλητής *rear* αλλάζει ως εξής:

$$\text{rear} \leftarrow \text{rear} + 1$$

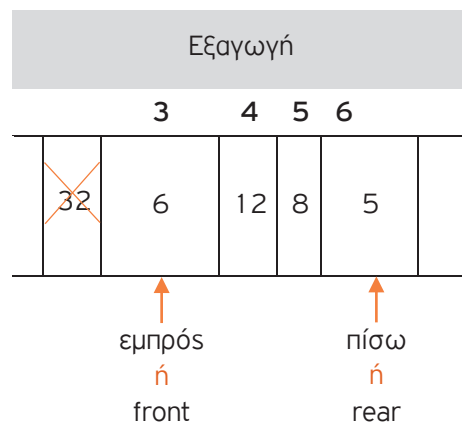
Κατά την εισαγωγή, πρώτα αυξάνουμε τον δείκτη *rear* κατά ένα και μετά εισάγουμε το στοιχείο στον πίνακα.



- Η **εξαγωγή** ενός στοιχείου γίνεται από το εμπρός άκρο της ουράς και η τιμή της μεταβλητής *front* αλλάζει ως εξής:

$$\text{front} \leftarrow \text{front} + 1$$

Κατά την εξαγωγή ενός στοιχείου, αυξάνεται ο δείκτης *front* κατά ένα (δείχνει στην επόμενη θέση του πίνακα) χωρίς στην πραγματικότητα να γίνεται καμία παρέμβαση στα περιεχόμενα του πίνακα (χωρίς να διαγράφεται κάποιο στοιχείο).



1.2.1 Παραδείγματα υλοποίησης ουράς με χρήση μονοδιάστατου πίνακα



Παράδειγμα 1 – Εισαγωγή και Εξαγωγή στοιχείων σε ουρά

1. Σε μια ουρά 10 θέσεων έχουν τοποθετηθεί διαδοχικά τα στοιχεία: Σ, Γ, Μ, Α, Δ στην 1η, 2η, 3η, 4η και 5η θέση αντίστοιχα.
 - i. Να προσδιορίσετε τις τιμές των δεικτών rear και front και να σχεδιάσετε την παραπάνω ουρά.
 - ii. Αν εφαρμόσετε τις παρακάτω λειτουργίες: **Εξαγωγή, Εξαγωγή, Εξαγωγή, Εισαγωγή Χ, Εισαγωγή Δ** και **Εξαγωγή**, ποιες είναι οι τιμές των δεικτών rear και front της ουράς και η τελική μορφή της;
2. Σε μια άδεια ουρά 10 θέσεων εισάγονται τα στοιχεία Ο, Σ, Λ, Τ, Ε. Με ποιον τρόπο πρέπει να εισαχθούν και να εξαχθούν τα στοιχεία, ώστε η έξοδος να εμφανίζει τα στοιχεία Τ, Ε, Λ, Ο, Σ;

Απάντηση

1)

1n	2n	3n	4n	5n	6n	7n	8n	9n	10n
Σ	Γ	M	A	Δ					
front					rear				

i)

```
front=1 kq| rear=5
```

ii) Η τελική μορφή της ουράς είναι:

1n	2n	3n	4n	5n	6n	7n	8n	9n	10n
				Δ	X	Δ			
				front		rear			

και οι τιμές της front και της rear γίνονται: front=5 και rear=7

2)

i) Η αρχική μορφή της ουράς είναι:

1n	2n	3n	4n	5n	6n	7n	8n	9n	10n
O	Σ	Λ	T	E					
front					rear				

front=1 and rear=5

ii) Με την εκτέλεση των λειτουργιών: **Εξαγωγή, Εξαγωγή, Εξαγωγή, Εισαγωγή Λ, Εισαγωγή Ο, Εισαγωγή Σ**, η τελική μορφή της ουράς γίνεται:

1n	2n	3n	4n	5n	6n	7n	8n	9n	10n
			T	E	Λ	O	Σ		
front				rear					

και οι τιμές της front και της rear γίνονται: front=4 και rear=8



Παράδειγμα 2 – Εισαγωγή στοιχείου σε ουρά με χρήση μονοδιάστατου πίνακα

Να αναπτύξετε τμήμα προγράμματος σε ΓΛΩΣΣΑ που υλοποιεί την εισαγωγή στοιχείου σε ουρά, με χρήση μονοδιάστατου πίνακα A, 10 θέσεων.

Απάντηση



Κώδικας σε ΓΛΩΣΣΑ [1.4]

```

1  ΓΡΑΨΕ 'Δώσε στοιχείο για εισαγωγή στην ουρά A:'
2  ΔΙΑΒΑΣΕ στοιχείο
3  ΑΝ rear = 10 ΤΟΤΕ
4      ΓΡΑΨΕ 'Γεμάτη ουρά'
5  ΑΛΛΙΩΣ_ΑΝ (front = 0 ΚΑΙ rear = 0) ΤΟΤΕ
6      front<- 1
7      rear<-1
8      A[rear]<-στοιχείο
9  ΑΛΛΙΩΣ
10     rear <- rear + 1
11     A[rear]<-στοιχείο
12 ΤΕΛΟΣ_ΑΝ
  
```



Παράδειγμα 3 – Εξαγωγή στοιχείου από ουρά με χρήση μονοδιάστατου πίνακα

Να αναπτύξετε τμήμα προγράμματος σε ΓΛΩΣΣΑ που υλοποιεί την εξαγωγή στοιχείου από ουρά, με χρήση μονοδιάστατου πίνακα A, 10 θέσεων.

Απάντηση



Κώδικας σε ΓΛΩΣΣΑ [1.5]

```

1  ΑΝ (front = 0 ΚΑΙ rear = 0) ΤΟΤΕ
2      ΓΡΑΨΕ 'Άδεια ουρά'
3  ΑΛΛΙΩΣ_ΑΝ (front = rear ) ΤΟΤΕ
4      ΓΡΑΨΕ 'Εξάγεται το στοιχείο:',A[front]
5      front <- 0
6      rear <- 0
7  ΑΛΛΙΩΣ
8      ΓΡΑΨΕ 'Εξάγεται το στοιχείο:',A[front]
9      front <- front + 1
10 ΤΕΛΟΣ_ΑΝ
  
```



Παράδειγμα 4 – Επιβίβαση και Αποβίβαση αυτοκινήτων σε πλοίο

Ένα οχηματαγωγό πλοίο με δύο διαφορετικές πόρτες, μία για την είσοδο και μία για την έξοδο των οχημάτων, χωρητικότητας 250 αυτοκινήτων, τα οποία δύνανται να τοποθετηθούν αποκλειστικά σε μία σειρά, εκτελεί το δρομολόγιο ΠΕΙΡΑΙΑΣ – ΑΙΓΙΝΑ. Τα οχήματα που **επιβιβάζονται πρώτα είναι και αυτά που θα αποβιβαστούν πρώτα**. Στο λιμάνι του Πειραιά προσέρχονται τα αυτοκίνητα για αναχώρηση.

Να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ το οποίο:

1. Να υλοποιεί μενού με τις επιλογές:
 1. Επιβίβαση
 2. Αποβίβαση
 3. Έξοδος
2. Στην περίπτωση που επιλεγεί η **Επιβίβαση** το πρόγραμμα θα διαβάζει τον αριθμό κυκλοφορίας καθενός από τα οχήματα που επιβιβάζονται στο πλοίο και θα τον καταχωρίζει στην ουρά ΟΧΗΜΑΤΑ. Κάθε φορά που επιβιβάζεται ένα όχημα να τυπώνεται το ερώτημα «**Υπάρχει όχημα για επιβίβαση; (Ν/Ο)**». Αν ο χρήστης απαντήσει Ν (=ΝΑΙ), τότε να επαναλαμβάνεται η διαδικασία επιβίβασης, ενώ αν απαντήσει Ο (=ΟΧΙ), τότε να σταματά η διαδικασία επιβίβασης και να επιστρέφει το πρόγραμμα στο μενού Επιλογής.
3. Στην περίπτωση που επιλεγεί η **Αποβίβαση** το πρόγραμμα θα εξάγει από την ουρά και θα εμφανίζει όλα τα αυτοκίνητα που αποβιβάστηκαν στην ΑΙΓΙΝΑ.

Απάντηση



Παρακάτω υλοποιείται το πρόγραμμα του παραδείγματος επιβίβασης/αποβίβασης σε οχηματαγωγό πλοίο, όπου η «πόρτα» για την είσοδο των οχημάτων στο πλοίο είναι διαφορετική από την «πόρτα» εξόδου των οχημάτων από αυτό.

Ανάλυση της λύσης

Σύμφωνα με την εκφώνηση της άσκησης πρέπει:

- Να υλοποιηθεί μια δομή δεδομένων FIFO (ουρά). Η υλοποίηση της ουράς θα πραγματοποιηθεί με μονοδιάστατο πίνακα 250 θέσεων (π[250]) όσο και το μέγιστο πλήθος επιβιβαζόμενων οχημάτων (250).
- Να δηλωθούν δύο βοηθητικές μεταβλητές (δείκτες) που δείχνουν την αρχή («**αρχ**») και το τέλος («**τέλος**») της ουράς. Οι αρχικές τιμές των δεικτών «**αρχ**» και «**τέλος**» θα είναι μηδέν.
- Στην περίπτωση της **επιβίβασης**, πρώτα θα αυξάνεται ο δείκτης «τέλος» κατά 1, για να δείχνει στην επόμενη κενή θέση της ουράς (τέλος<-τέλος+1). Σε αυτή την κενή θέση θα καταχωρίζεται ο αριθμός κυκλοφορίας («**αρ**») του επιβιβαζόμενου οχήματος (π[τέλος]<-αρ).
- Στην περίπτωση της **αποβίβασης**, πρώτα θα αποβιβάζεται το όχημα και μετά ο δείκτης «**αρχ**» θα αυξάνεται κατά 1 (αρχ<-αρχ+1), έτσι ώστε να δείχνει στη θέση που αντιστοιχίζεται στο επόμενο όχημα προς αποβίβαση.
- Να δηλωθούν και οι μεταβλητές: **επ1** (επιλογή μενού), **πλ** (μετρητής αποβιβαζόμενων οχημάτων), **επ2** (μεταβλητή δήλωσης οχήματος για επιβίβαση), **αρ** (αριθμός κυκλοφορίας οχήματος, μέσω του οποίου καταχωρίζεται-δηλώνεται η επιβίβασή του), **π[250]** (πίνακας 250 θέσεων, που φαίνεται το μέγιστο πλήθος των οχημάτων που μπορούν να χωρέσουν στο πλοίο).

- Να εμφανίζεται ένα μενού επιλογών με τις επιλογές: **1.** «Επιβίβαση», **2.** «Αποβίβαση», **3.** «Εξοδος» (δομή επανάληψης «**ΜΕΧΡΙΣ_ΟΤΟΥ...**», με τη βοήθεια της μεταβλητής «**επ1**» η οποία θα παίρνει τις τιμές 1 ή 2 ή 3).
- Στην περίπτωση της **επιβίβασης** («**ΑΝ επ1=1 ΤΟΤΕ**»):
 - ο Αρχικά, γίνεται έλεγχος αν το πλοίο είναι γεμάτο («**ΑΝ τελος=250 ΤΟΤΕ**»). Στην περίπτωση αυτή εμφανίζεται κατάλληλο μήνυμα («**ΓΡΑΨΕ 'Το πλοίο είναι πλήρες και δε χωρά άλλα οχήματα'**»)
 - ο Στη συνέχεια, θα πρέπει να ελεγχθεί αν υπάρχει όχημα για επιβίβαση. Ο έλεγχος πραγματοποιείται μέσω της μεταβλητής «**επ2**» («**ΔΙΑΒΑΣΕ επ2, ΑΝ επ2 <>'N'...**»).
 - ο Αν υπάρχει όχημα προς επιβίβαση, θα πρέπει να ελέγχεται αν είναι κενό το πλοίο/ουρά. Αν είναι κενό («**ΑΝ (αρχ=0 ΚΑΙ τελος=0) ΤΟΤΕ**»), αυξάνουμε τους δείκτες «**αρχ**» και «**τέλος**» για να δείχνουν στην πρώτη κενή θέση του πλοίου/ουράς («**αρχ<-1, τελος<-1**») και μετά γίνεται καταχώριση του αριθμού κυκλοφορίας του επιβιβαζόμενου οχήματος στη θέση αυτή (πρώτη θέση του πλοίου/ουράς) («**π[τελος]<- αρχ**»).
Στην περίπτωση που το πλοίο/ουρά δεν είναι κενό, αυξάνεται πρώτα η τιμή του πίσω δείκτη της ουράς («**τελος<-τελος+1**»), για να δείχνει σε κενή θέση κι έπειτα καταχωρίζεται σε αυτή ο αριθμός κυκλοφορίας του επιβιβαζόμενου οχήματος. Η διαδικασία αυτή επαναλαμβάνεται όσο υπάρχουν οχήματα προς επιβίβαση και το πλοίο δεν έχει γεμίσει («**ΜΕΧΡΙΣ_ΟΤΟΥ τελος =250 Η επ2='Ο' Η επ2='ο'**»).
 - ο Στην περίπτωση που διακοπεί η επαναληπτική διαδικασία της επιβίβασης λόγω «γεμίματος» του πλοίου («**ΑΝ τελος=250 ΤΟΤΕ**»), εμφανίζεται κατάλληλο μήνυμα («**ΓΡΑΨΕ 'Το πλοίο είναι πλήρες και δε χωρά άλλα οχήματα'**»).
- Στην περίπτωση της **αποβίβασης** («**ΑΛΛΙΩΣ_ΑΝ επ1=2 ΤΟΤΕ**»):
 - ο Θα πρέπει πρώτα να ελεγχθεί αν το πλοίο/ουρά είναι κενό («**ΑΝ (αρχ=0 ΚΑΙ τελος=0) ΤΟΤΕ**») και στην περίπτωση αυτή να εμφανίζεται κατάλληλο μήνυμα («**ΓΡΑΨΕ 'Το πλοίο είναι άδειο'**»)
 - ο Έπειτα, θα πρέπει να γίνει έλεγχος της περίπτωσης που το πλοίο/ουρά έχει ένα μόνο όχημα («**ΑΛΛΙΩΣ_ΑΝ αρχ=τελος ΤΟΤΕ**»). Στην περίπτωση αυτή πρώτα αποβιβάζεται το όχημα από το πλοίο («εξάγεται» από την ουρά, «**ΓΡΑΨΕ 'Αποβιβάζεται το μοναδικό όχημα:', π[αρχ]**») και μετά μηδενίζουμε τους δείκτες («**αρχ<-0, 'τέλος<-0**»), αφού το πλοίο/ουρά είναι πλέον κενό.
Σημειώνεται ότι για λόγους παρακολούθησης των τιμών του πίνακα, στη θέση του πίνακα που κατείχε το αποβιβαζόμενο όχημα καταχωρίζεται ο κενός χαρακτήρας («**π[αρχ]<-' '**»).
 - ο Αν υπάρχουν περισσότερα από ένα οχήματα για αποβίβαση, μέσα σε μια επαναληπτική διαδικασία πρώτα εμφανίζεται το όχημα που αποβιβάζεται («**ΓΡΑΨΕ 'Αποβιβάζεται το όχημα:', π[αρχ]**») κι έπειτα ο μπροστινός δείκτης «**αρχ**» αυξάνεται κατά 1 («**αρχ<-αρχ+1**») και δείχνει στη θέση του επόμενου οχήματος προς αποβίβαση. Η επαναληπτική διαδικασία αποβίβασης ολοκληρώνεται, όταν πλέον δεν υπάρχει κανένα όχημα για αποβίβαση, δηλαδή όταν ο δείκτης «**αρχ**» γίνει μεγαλύτερος από τον πίσω δείκτη «**τέλος**» της ουράς («**ΜΕΧΡΙΣ_ΟΤΟΥ (αρχ>τελος)**»).
 - ο Τέλος, με την ολοκλήρωση της αποβίβασης όλων των οχημάτων, εκτυπώνεται με κατάλληλο μήνυμα το πλήθος αποβιρασθέντων οχημάτων στην ΑΙΓΙΝΑ («**ΓΡΑΨΕ 'Οχήματα που αποβιβάστηκαν στην ΑΙΓΙΝΑ: ', πλ**»).

- Ο Σε κάθε νέα εκκίνηση αποβίβασης οχημάτων, ο μετρητής «πλ» πρέπει να μηδενίζεται, για να είναι σωστό το σύνολο των αποβιbasθέντων οχημάτων της τρέχουσας αποβίbasης. Π.χ. εκτελείται το πρόγραμμα και επιβιβάζονται 15 οχήματα, τα οποία στη συνέχεια αποβιβάζονται. Στη συνέχεια, επιλέγεται, από το μενού, νέα επιβίbasη 30 οχημάτων, τα οποία στη συνέχεια αποβιβάζονται. Αν ο μετρητής «πλ» δεν μηδενίζεται σε κάθε νέα εκκίνηση αποβίbasης, στην πρώτη αποβίbasη θα είχε την τιμή «15» και στη δεύτερη την τιμή «45» και όχι την τιμή «30» που είναι η σωστή. Δηλαδή, στη δεύτερη αποβίbasη οχημάτων το πρόγραμμα θα τύπωνε λανθασμένα «Οχήματα που αποβιbasήθηκαν στην ΑΙΓΙΝΑ: 45» και όχι το σωστό «Οχήματα που αποβιbasήθηκαν στην ΑΙΓΙΝΑ: 30» («**ΓΡΑΨΕ** 'Οχήματα που αποβιbasήθηκαν στην ΑΙΓΙΝΑ: ', πλ»).
- Επίσης, αν επιλεχθεί νέα επιβίbasη μετά την ολοκλήρωση κάθε αποβίbasης, θα πρέπει η ουρά να γεμίζει από την αρχή. Κάτι τέτοιο επιτυγχάνεται με τον μηδενισμό των τιμών των δύο δεικτών, καθώς παίρνουν την ίδια τιμή («αρχ <- 0», «τελος <- 0»).

Ακολουθεί η υλοποίηση του προγράμματος σε ΓΛΩΣΣΑ.



Κώδικας σε ΓΛΩΣΣΑ [1.6]

ΠΡΟΓΡΑΜΜΑ πλοίο2

ΜΕΤΑΒΛΗΤΕΣ

ΑΚΕΡΑΙΕΣ: αρχ, τελος, επ1, πλ

ΧΑΡΑΚΤΗΡΕΣ: επ2, αρ, π[250]

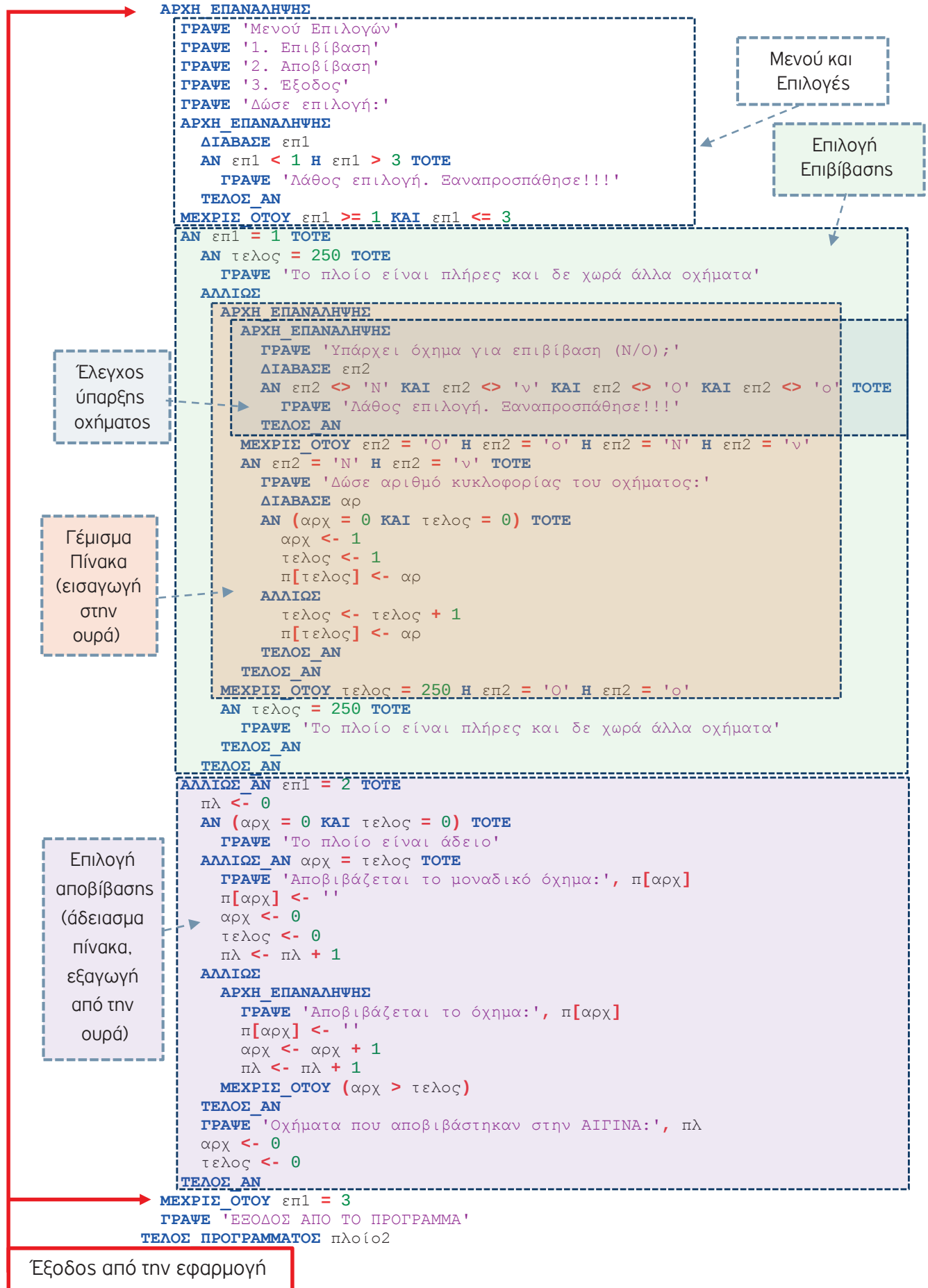
ΑΡΧΗ

αρχ <- 0

τελος <- 0

Δήλωση Μεταβλητών

Αρχικοποίηση
Μεταβλητών



```

1  ΠΡΟΓΡΑΜΜΑ πλοίο2
2  ΜΕΤΑΒΛΗΤΕΣ
3      ΑΚΕΡΑΙΕΣ: αρχ, τελος, επ1, πλ
4      ΧΑΡΑΚΤΗΡΕΣ: επ2, αρ, π[250]
5  ΑΡΧΗ
6      αρχ <- 0
7      τελος <- 0
8      ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
9          ΓΡΑΨΕ 'Μενού Επιλογών'
10         ΓΡΑΨΕ '1. Επιβίβαση'
11         ΓΡΑΨΕ '2. Αποβίβαση'
12         ΓΡΑΨΕ '3. Έξοδος'
13         ΓΡΑΨΕ 'Δώσε επιλογή:'
14         ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
15             ΔΙΑΒΑΣΕ επ1
16             ΑΝ επ1 < 1 Η επ1 > 3 ΤΟΤΕ
17                 ΓΡΑΨΕ 'Λάθος επιλογή. Ξαναπροσπάθησε!!!'
18             ΤΕΛΟΣ_ΑΝ
19             ΜΕΧΡΙΣ_ΟΤΟΥ επ1 >= 1 ΚΑΙ επ1 <= 3
20             ΑΝ επ1 = 1 ΤΟΤΕ
21                 ΑΝ τελος = 250 ΤΟΤΕ
22                     ΓΡΑΨΕ 'Το πλοίο είναι πλήρες και δε χωρά άλλα οχήματα'
23                 ΑΛΛΙΩΣ
24                     ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
25                         ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
26                             ΓΡΑΨΕ 'Υπάρχει όχημα για επιβίβαση (N/O);'
27                             ΔΙΑΒΑΣΕ επ2
28                             ΑΝ επ2<>'N' ΚΑΙ επ2<>'ν' ΚΑΙ επ2<>'Ο' ΚΑΙ επ2<>'ο' ΤΟΤΕ
29                                 ΓΡΑΨΕ 'Λάθος επιλογή. Ξαναπροσπάθησε!!!'
30                             ΤΕΛΟΣ_ΑΝ
31                             ΜΕΧΡΙΣ_ΟΤΟΥ επ2='Ο' Η επ2='ο' Η επ2='N' Η επ2='ν'
32                             ΑΝ επ2 = 'N' Η επ2 = 'ν' ΤΟΤΕ
33                                 ΓΡΑΨΕ 'Δώσε αριθμό κυκλοφορίας του οχήματος:'
34                                 ΔΙΑΒΑΣΕ αρ
35                                 ΑΝ (αρχ = 0 ΚΑΙ τελος = 0) ΤΟΤΕ
36                                     αρχ <- 1
37                                     τελος <- 1
38                                     π[τελος] <- αρ
39                                 ΑΛΛΙΩΣ
40                                     τελος <- τελος + 1

```

```

41         π[τελος] <- αρ
42         ΤΕΛΟΣ_ΑΝ
43         ΤΕΛΟΣ_ΑΝ
44         ΜΕΧΡΙΣ_ΟΤΟΥ τελος = 250 Η επ2 = 'Ο' Η επ2 = 'ο'
45         ΑΝ τελος = 250 ΤΟΤΕ
46             ΓΡΑΨΕ 'Το πλοίο είναι πλήρες και δε χωρά άλλα οχήματα'
47         ΤΕΛΟΣ_ΑΝ
48         ΤΕΛΟΣ_ΑΝ
49     ΑΛΛΙΩΣ_ΑΝ επ1 = 2 ΤΟΤΕ
50         πλ <- 0
51         ΑΝ (αρχ = 0 ΚΑΙ τελος = 0) ΤΟΤΕ
52             ΓΡΑΨΕ 'Το πλοίο είναι άδειο'
53         ΑΛΛΙΩΣ_ΑΝ αρχ = τελος ΤΟΤΕ
54             ΓΡΑΨΕ 'Αποβιβάζεται το μοναδικό όχημα:', π[αρχ]
55             π[αρχ] <- ''
56             αρχ <- 0
57             τελος <- 0
58             πλ <- πλ + 1
59         ΑΛΛΙΩΣ
60             ΑΡΧΗ_ΕΠΑΝΑΛΗΨΗΣ
61                 ΓΡΑΨΕ 'Αποβιβάζεται το όχημα:', π[αρχ]
62                 π[αρχ] <- ''
63                 αρχ <- αρχ + 1
64                 πλ <- πλ + 1
65             ΜΕΧΡΙΣ_ΟΤΟΥ (αρχ > τελος)
66             ΤΕΛΟΣ_ΑΝ
67             ΓΡΑΨΕ 'Οχήματα που αποβιβάστηκαν στην ΑΙΓΙΝΑ:', πλ
68             αρχ <- 0
69             τελος <- 0
70             ΤΕΛΟΣ_ΑΝ
71         ΜΕΧΡΙΣ_ΟΤΟΥ επ1 = 3
72         ΓΡΑΨΕ 'ΕΞΟΔΟΣ ΑΠΟ ΤΟ ΠΡΟΓΡΑΜΜΑ'
73     ΤΕΛΟΣ_ΠΡΟΓΡΑΜΜΑΤΟΣ πλοίο2

```

1.2.2 Ερωτήσεις - Ασκήσεις

Ε. 1: Να δώσετε παραδείγματα ουράς από την καθημερινή ζωή.

Ε. 2: Δίνεται η επόμενη ακολουθία αριθμών : 4, 8, 2, 5, 9, 13.

1. Ποια λειτουργία θα χρησιμοποιήσετε για την τοποθέτηση των αριθμών σε ουρά;
2. Να σχεδιάσετε την ουρά έπειτα από την τοποθέτηση των αριθμών.
3. Ποια λειτουργία θα χρησιμοποιήσετε για την εξαγωγή των αριθμών από την ουρά;
4. Πόσες φορές θα πρέπει να εκτελεστεί η προηγούμενη λειτουργία στην ουρά για να εξαχθεί ο αριθμός 5;

Ε. 3:

1. Σε μια ουρά 10 θέσεων έχουν τοποθετηθεί διαδοχικά τα στοιχεία: X, A, B, A, P στην 1η, 2η, 3η, 4η και 5η θέση αντίστοιχα.
 - i. Να προσδιορίσετε τις τιμές των δεικτών rear και front της παραπάνω ουράς και να τη σχεδιάσετε.
 - ii. Αν εφαρμόσουμε τις ακόλουθες λειτουργίες: **Εξαγωγή, Εξαγωγή, Εξαγωγή, Εισαγωγή X, Εισαγωγή Δ** και **Εξαγωγή** ποιες είναι τις τιμές των δεικτών rear και front της παραπάνω ουράς και ποια η τελική μορφή της ουράς;
2. Σε μια κενή ουρά 10 θέσεων εισάγουμε τα στοιχεία K, Φ, Ι, Α,Ρ. Με ποιον τρόπο πρέπει να «εισαχθούν» και να «εξαχθούν» τα στοιχεία, ώστε να έχουμε ως έξοδο τα δεδομένα A, P, X, H.

Ε. 4: Σε μια τράπεζα χρησιμοποιείται αυτόματο ηλεκτρονικό μηχάνημα που το χειρίζονται οι πελάτες, οι ταμίες και ο διευθυντής της τράπεζας. Κάθε ένας από τους χειριστές του μηχανήματος έχει δικαιώματα χρήσης συγκεκριμένων πλήκτρων του πληκτρολογίου.

Ο πελάτης το «Π», οι ταμίες το «1» ή το «2» ή το «3» ή το «4» αναλόγως της θέσης του ταμείου που εργάζονται και ο διευθυντής το πλήκτρο «Δ». Κατά την είσοδό του, ο κάθε πελάτης πατάει το πλήκτρο «Π» και εκτυπώνεται ένα χαρτί, στο οποίο αναγράφεται το νούμερο που έχει στην ουρά από την αρχή της ημέρας.

Η τράπεζα έχει 4 ταμεία, όπου όταν ο ταμίας εξυπηρετεί έναν πελάτη, πατάει το νούμερο του ταμείου του, «1» ή «2» ή «3» ή «4».

Ο διευθυντής της τράπεζας, πατώντας το κουμπί «Δ», σταματά τη διαδικασία εξυπηρέτησης των πελατών και μπορεί να δει το σύνολο των πελατών που έχουν ήδη εξυπηρετηθεί από το κάθε ταμείο. Ο μέγιστος αριθμός πελατών που μπορεί να εξυπηρετήσει η τράπεζα είναι 1.000 πελάτες.

Λαμβάνοντας υπόψη ότι ο πρώτος πελάτης εξυπηρετείται πρώτος και ο τελευταίος εξυπηρετείται τελευταίος, να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ, όπου:

1. Να υπάρχει μενού επιλογής:
Π. Πελάτης Τ. Ταμίας Δ. Διευθυντής
 2. Στην περίπτωση που επιλεγθεί από το μενού το **Π. Πελάτης**, το πρόγραμμα εκτυπώνει το νούμερο που έχει στην ουρά (από την αρχή της ημέρας).
 3. Στην περίπτωση που επιλεγθεί από το μενού το **Τ. Ταμίας**, ο/η αρμόδιος/-α υπάλληλος επιλέγει το νούμερο του ταμείου που του/της αντιστοιχεί: «**1**» ή «**2**» ή «**3**» ή «**4**» και ο πελάτης διαγράφεται από την ουρά.
 4. Στην περίπτωση που επιλεγθεί από το μενού το **Δ. Διευθυντής**, σταματά η διαδικασία εξυπηρέτησης και το πρόγραμμα τυπώνει το νούμερο του ταμείου που εξυπηρέτησε τους περισσότερους πελάτες.
- Στο πρόγραμμα να γίνεται έλεγχος των δεδομένων εισόδου.

Ε. 5: Μια αεροπορική εταιρεία εκτελεί το δρομολόγιο Αθήνα – Θεσσαλονίκη κατά την περίοδο του Σεπτεμβρίου. Λόγω της Δ.Ε.Θ. υπάρχει αυξημένη ζήτηση και η εταιρεία διατηρεί λίστα αναμονής για τους επιβάτες που δεν πρόλαβαν να κλείσουν εισιτήριο, ώστε αν προκύψει κάποια ακύρωση, να ενημερώσει τον πρώτο στη σειρά πελάτη που εισήχθη στη λίστα αναμονής προκειμένου να κλείσει εισιτήριο. Η λίστα αναμονής δεν μπορεί να περιλαμβάνει περισσότερα από 10 ονόματα.

Να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ το οποίο:

1. Να υπάρχει μενού επιλογής: **1. ΕΓΓΡΑΦΗ 2. ΑΚΥΡΩΣΗ 3. ΤΕΛΟΣ**.
2. Αν ο χρήστης επιλέξει την τιμή «**1.ΕΓΓΡΑΦΗ**», τότε θα ζητείται το όνομα του χρήστη και θα καταχωρίζεται στη λίστα αναμονής, εφόσον η λίστα αναμονής δεν έχει γεμίσει. Διαφορετικά, θα εμφανίζεται το μήνυμα: «Η λίστα αναμονής είναι πλήρης».
3. Αν ο χρήστης επιλέξει την τιμή «**2.ΑΚΥΡΩΣΗ**», τότε κάποιος από τους επιβάτες της πτήσης έχει ακυρώσει την κράτησή του, συνεπώς, το πρόγραμμα θα πρέπει να εμφανίσει το όνομα του ατόμου που είναι το πρώτο διαθέσιμο στη λίστα αναμονής. Αν δεν υπάρχουν άτομα στη λίστα αναμονής, εμφανίζεται το μήνυμα «Η λίστα αναμονής είναι άδεια».
4. Η παραπάνω διαδικασία επαναλαμβάνεται μέχρι ο χρήστης να επιλέξει την τιμή «**3.ΤΕΛΟΣ**». Το πρόγραμμα εμφανίζει το πλήθος των ατόμων που κατάφεραν να κάνουν κράτηση μέσα από την λίστα αναμονής, καθώς και το μέγιστο πλήθος των ατόμων που περίμεναν στην ουρά αναμονής.

Στο πρόγραμμα να γίνεται έλεγχος εγκυρότητας των τιμών που πληκτρολογούνται.

Ε. 6: Σε ένα ταχυδρομικό κατάστημα, οι πελάτες εξυπηρετούνται με βάση τη σειρά άφιξής τους σε αυτό. Το ταχυδρομικό κατάστημα έχει ένα ταμείο και ο μέσος χρόνος εξυπηρέτησης κάθε πελάτη είναι 3 λεπτά. Η ουρά αναμονής στο κατάστημα δεν μπορεί να ξεπερνά τα 30 άτομα.

Να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ το οποίο:

1. Να δέχεται σαν είσοδο από τον χρήστη μία εκ των δύο τιμών εισαγωγής: «**1.ΕΙΣΑΓΩΓΗ**» ή «**2.ΕΠΟΜΕΝΟΣ**» (με έλεγχο εγκυρότητας).

2. Αν δοθεί η τιμή «**1.ΕΙΣΑΓΩΓΗ**», τότε το πρόγραμμα να διαβάζει το ονοματεπώνυμο του πελάτη και αμέσως μετά να εμφανίζει το πλήθος των ατόμων που περιμένουν πριν από αυτόν, εκτός αν η ουρά αναμονής είναι γεμάτη, οπότε να εμφανίζει το μήνυμα «Το κατάστημα γέμισε. Παρακαλούμε ελάτε άλλη φορά».
3. Αν δοθεί η τιμή «**2.ΕΠΟΜΕΝΟΣ**», τότε το πρόγραμμα να εμφανίζει το ονοματεπώνυμο του πελάτη προς εξυπηρέτηση.
4. Η παραπάνω διαδικασία να επαναλαμβάνεται μέχρι να εξυπηρετηθούν όλοι οι πελάτες.
5. Στο τέλος το πρόγραμμα να εμφανίζει το πλήθος των ατόμων που εξυπηρετήθηκαν, καθώς και τον μέσο χρόνο αναμονής των πελατών.

Ε. 7: Ένας εκτυπωτής χρησιμοποιεί μια ουρά εκτύπωσης για να τοποθετεί σε αυτήν τα αρχεία που έχουν σταλεί προς εκτύπωση με τη σειρά που αυτά στάλθηκαν. Κάθε φορά εκτυπώνει το αρχείο που βρίσκεται στην αρχή της ουράς εκτύπωσης, το οποίο και εξάγει. Λόγω της περιορισμένης μνήμης του εκτυπωτή, θεωρούμε ότι στην ουρά μπορούν να εισαχθούν το πολύ 15 αρχεία.

Να αναπτύξετε πρόγραμμα σε ΓΛΩΣΣΑ το οποίο:

1. Να διαβάζει επαναληπτικά, με έλεγχο εγκυρότητας, το γράμμα “N” που καθορίζει την έλευση νέου αρχείου ή το γράμμα “E” που δηλώνει την προσπάθεια εκτύπωσης ενός αρχείου.
2. Κατά την έλευση ενός αρχείου, διαβάζει το όνομά του και εξετάζει αν υπάρχει ο διαθέσιμος χώρος στην ουρά και το αρχείο καταχωρίζεται σε αυτήν με τη διαδικασία της εισαγωγής. Στην περίπτωση που δεν υπάρχει χώρος, εμφανίζεται το μήνυμα «Η ουρά γέμισε. Δε μπορεί να εκτυπωθεί το αρχείο».
3. Όταν ο χρήστης δώσει το γράμμα “E”, εξετάζει αν υπάρχουν αρχεία προς εκτύπωση και στην περίπτωση αυτή εξάγεται το κατάλληλο αρχείο εμφανίζοντας τη λέξη «Εκτύπωση» ακολουθούμενη από το όνομα του αρχείου που τυπώνεται.
4. Η επαναληπτική διαδικασία ολοκληρώνεται, όταν εκτυπωθούν όλα τα αρχεία που έχουν τοποθετηθεί στην ουρά.
5. Μετά το τέλος της διαδικασίας, το πρόγραμμα εμφανίζει τον συνολικό αριθμό των αρχείων που εκτυπώθηκαν.

Ε. 8: Χαρακτηρίστε τις παρακάτω προτάσεις ως Σωστές ή Λάθος. Στην περίπτωση που πιστεύετε ότι είναι λανθασμένες δικαιολογήστε την επιλογή σας και σκεφτείτε ποια θα μπορούσε να είναι η αντίστοιχη σωστή πρόταση.

α/α	Προτάσεις	Σ	Λ
1	Για την υλοποίηση της ουράς μπορεί να χρησιμοποιηθεί πίνακας.	☐	☐
2	Κατά την εισαγωγή ενός στοιχείου σε ουρά, αυτό τοποθετείται στο μπροστινό άκρο της.	☐	☐
3	Σε μια ουρά κάθε στοιχείο της εξάγεται από το μπροστινό άκρο της.	☐	☐
4	Η απώθηση είναι μια από τις λειτουργίες της ουράς.	☐	☐
5	Η εισαγωγή και η εξαγωγή είναι οι δύο βασικές λειτουργίες της ουράς.	☐	☐
6	Στην ουρά το στοιχείο που μπαίνει πρώτο βγαίνει και πρώτο.	☐	☐
7	Η υλοποίηση της ουράς χρησιμοποιεί μία μεταβλητή-δείκτη για την εκτέλεση των δύο βασικών λειτουργιών της.	☐	☐
8	Η λειτουργία της εξαγωγής μπορεί να εκτελεστεί σε μια γεμάτη ουρά.	☐	☐

1.3 Άλλες δομές δεδομένων

Κοινό γνώρισμα των παραπάνω δομών που υλοποιήθηκαν με χρήση μονοδιάστατου πίνακα είναι ότι οι διαδοχικοί κόμβοι αποθηκεύονται σε συνεχόμενες θέσεις της κύριας μνήμης. Στην ενότητα αυτή παρουσιάζεται η περίπτωση τριών σημαντικών δομών δεδομένων, στις οποίες οι κόμβοι δεν είναι απαραίτητο να κατέχουν συνεχόμενες θέσεις μνήμης. Οι δομές αυτές ανήκουν στην κατηγορία των δυναμικών δομών δεδομένων και πρόκειται για τις λίστες, τα δένδρα και τους γράφους.

1.3.1 Λίστες

Η έννοια της λίστας συναντάται αρκετά συχνά στην καθημερινότητά μας. Πόσες φορές δεν έχουμε καταρτίσει λίστες με τα αγαπημένα μας τραγούδια, τις επαφές μας, τα ψώνια που θέλουμε να κάνουμε ή ακόμα και λίστες με επιθυμίες και όνειρα; Η λίστα δεν είναι τίποτα άλλο παρά μία συλλογή από αντικείμενα του ίδιου τύπου. Μπορούμε να έχουμε δηλαδή λίστες από λέξεις, από ονόματα αλλά και από αριθμούς.

Πώς όμως θα μπορούσατε να αποθηκεύσετε έναν συγκεκριμένο αριθμό στοιχείων του ίδιου τύπου, για παράδειγμα: τα μέρη που θα θέλατε να επισκεφτείτε; Μία από τις δομές δεδομένων που έχετε δει μέχρι τώρα και που θα μπορούσατε να χρησιμοποιήσετε για να αποθηκεύσετε τα στοιχεία αυτά είναι οι πίνακες. Πολύ απλά και εύκολα μπορείτε να τους δημιουργήσετε και να προσπελάσετε άμεσα οποιοδήποτε στοιχείο τους.

Το γεγονός, όμως, ότι το μέγεθος ενός πίνακα παραμένει σταθερό (στατική δομή δεδομένων), δυσχεραίνει την προσθήκη και τη διαγραφή στοιχείων. Θεωρείστε την περίπτωση που θέλετε να προσθέσετε στον πίνακα και άλλα μέρη στα οποία θέλετε να ταξιδέψετε. Τι θα κάνετε αν ο πίνακας είναι γεμάτος; Μια πρώτη απάντηση θα ήταν να δημιουργήσουμε έναν νέο μεγαλύτερο πίνακα και να αντιγράψουμε σε αυτόν τα στοιχεία του προηγούμενου πίνακα. Έτσι, θα είχαμε μεγαλύτερο χώρο για να προσθέσουμε και νέα στοιχεία. Θεωρείτε ότι αυτή είναι η ιδανική λύση, όταν η προσθήκη και η διαγραφή στοιχείων αποτελεί συχνό φαινόμενο;

Ας μελετήσουμε μία άλλη λύση, δηλαδή μία νέα δυναμική δομή δεδομένων, τις **συνδεδεμένες λίστες**, που απαντάει σε αυτές τις προκλήσεις. Αρχικά, ας θεωρήσουμε το πλέγμα των κελιών που παρουσιάζεται στην Εικόνα 1.3.1.α ως τη μνήμη του υπολογιστή. Σε κάθε κελί αντιστοιχίζεται μία διεύθυνση. Για να προσπελάσουμε ένα οποιαδήποτε κελί θα πρέπει να γνωρίζουμε τη διεύθυνσή του. Στην Εικόνα 1.3.1.α παρουσιάζεται ο χώρος που καταλαμβάνει ο πίνακας Α με το γαλάζιο χρώμα. Ο πίνακας αυτός αποτελείται από έξι στοιχεία τα οποία είναι αποθηκευμένα σε συνεχόμενες θέσεις μνήμης.

A[1] A[2] A[3] A[4] A[5] A[6]

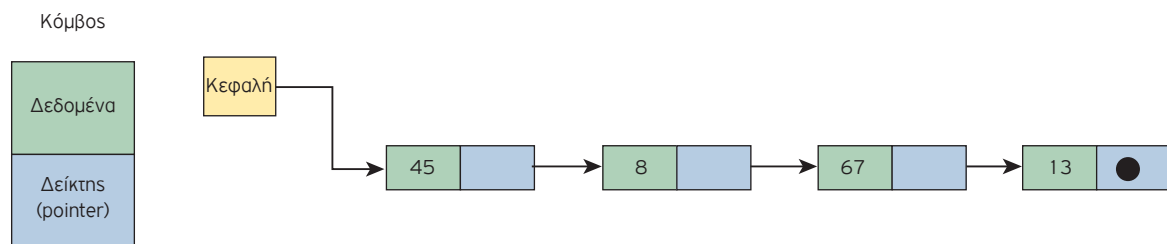
Εικόνα 1.3.1. α: Ο πίνακας Α στη μνήμη

β: Με το χρώμα κόκκινο αναπαριστάσουμε τα ελεύθερα κελιά

Συνήθως όμως, η μνήμη του υπολογιστή δεν είναι τόσο καθαρή και τακτοποιημένη. Στην Εικόνα 1.3.1.β φαίνονται τα διάσπαρτα (μη συνεχόμενα) ελεύθερα κελιά με το κόκκινο χρώμα. Στην προκειμένη περίπτωση, δεν μπορείτε να «τοποθετήσετε» έναν πίνακα στη μνήμη, διότι δεν υπάρχουν συνεχόμενες ελεύθερες θέσεις. Μπορείτε, όμως, να χρησιμοποιήσετε τις διάσπαρτες αυτές ελεύθερες θέσεις για να αποθηκεύσετε τα δεδομένα σας, αν θεωρήσετε ότι αποτελούν «στοιχεία» μίας συνδεδεμένης λίστας.

Η συνδεδεμένη λίστα αποτελείται από μία σειρά από κόμβους, που συνήθως βρίσκονται σε απομακρυσμένες θέσεις μνήμης. Κάθε κόμβος αποτελείται από δύο κύρια τμήματα (Εικόνα 1.3.2.α). Το πρώτο τμήμα περιέχει τα δεδομένα και το δεύτερο τμήμα φιλοξενεί τη διεύθυνση του επόμενου κόμβου με τον οποίο συνδέεται ή όπως αλλιώς θα λέγαμε στη γλώσσα των δομών δεδομένων, το δεύτερο τμήμα περιέχει έναν δείκτη (pointer) που δείχνει στον επόμενο κόμβο.

Το πεδίο Δεδομένα μπορεί να περιέχει μία ή περισσότερες αλφαριθμητικές ή αριθμητικές πληροφορίες. Ο δείκτης (pointer) είναι ένας ιδιαίτερος τύπος δεδομένων που προσφέρεται από τις περισσότερες σύγχρονες γλώσσες προγραμματισμού. Ο δείκτης δε λαμβάνει αριθμητικές τιμές όπως ακέραιες, πραγματικές κ.ά., αλλά οι τιμές του είναι διευθύνσεις στην κύρια μνήμη και χρησιμοποιείται ακριβώς για τη σύνδεση των διαφόρων στοιχείων μιας δομής, που είναι αποθηκευμένα σε μη συνεχόμενες θέσεις μνήμης.



Εικόνα 1.3.2. α: Αναπαράσταση κόμβου

β: Αναπαράσταση απλά συνδεδεμένης λίστας με 4 κόμβους

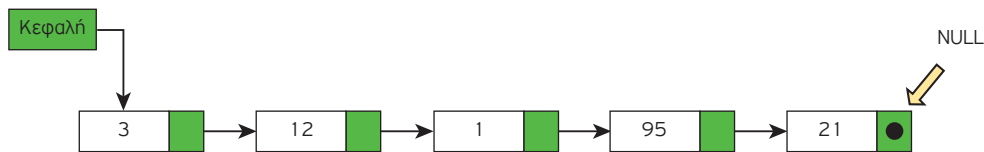
Ένα παράδειγμα απλά συνδεδεμένης λίστας με τέσσερις κόμβους, που περιέχουν ακραίους, δίνεται στην Εικόνα 1.3.2.β. Ο δείκτης του τελευταίου κόμβου της λίστας έχει ως τιμή το NULL (κενό) και τον αναπαριστούμε συμβολικά με το σύμβολο «●». Οι δείκτες των υπολοίπων κόμβων περιέχουν τη διεύθυνση του επόμενου κόμβου αντίστοιχα και τους απεικονίζουμε συμβολικά με ένα βέλος για να υποδηλώσουμε τη σύνδεση μεταξύ του προηγούμενου και του επόμενου κόμβου. Κάθε λίστα συνοδεύεται από έναν δείκτη με το όνομα «Κεφαλή» (head), που δείχνει στον πρώτο κόμβο της λίστας, δηλαδή περιέχει τη διεύθυνση του πρώτου κόμβου της λίστας.



Μία (απλά) **συνδεδεμένη λίστα (linked list)** είναι ένα σύνολο κόμβων διατεταγμένων γραμμικά (ο ένας μετά τον άλλο). Κάθε κόμβος περιέχει εκτός από τα **δεδομένα** του και έναν **δείκτη** που δείχνει προς τον επόμενο κόμβο.

Ο δείκτης του τελευταίου κόμβου δε δείχνει σε κάποιον κόμβο (δείκτης στο κενό). Για να το δηλώσουμε αυτό λέμε ότι το πεδίο δείκτη του τελευταίου κόμβου έχει την τιμή **NULL**.

Για να προσπελάσουμε τους κόμβους της λίστας χρειάζεται να γνωρίζουμε τη διεύθυνση (θέση στη μνήμη) του πρώτου κόμβου της λίστας. Η διεύθυνση αυτή αποθηκεύεται σε μία ειδική μεταβλητή που την ονομάζουμε συνήθως **Κεφαλή (Head)**.



Εικόνα 1.3.3. Μία απλά συνδεδεμένη λίστα με 5 κόμβους.

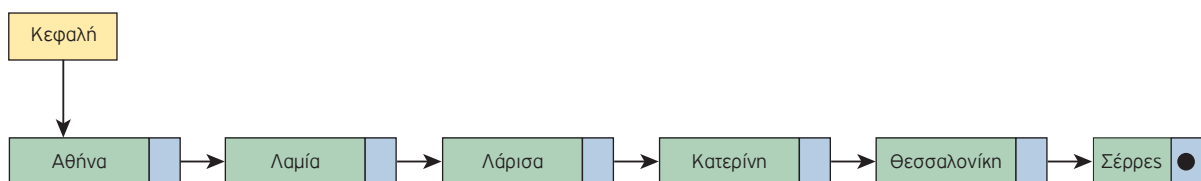
Οι κόμβοι μιας λίστας δεν έχουν ονόματα. Γνωρίζουμε μόνο τις διευθύνσεις τους, που είναι αποθηκευμένες στους προηγούμενους κόμβους και αυτές θα αξιοποιήσουμε για να τους προσπελάσουμε. Επομένως, αν θελήσουμε να έχουμε πρόσβαση στον τέταρτο κόμβο μιας λίστας, για να επεξεργαστούμε τα δεδομένα που περιέχει, θα πρέπει να ξεκινήσουμε από τον πρώτο κόμβο της λίστας, η διεύθυνση του οποίου περιέχεται στον δείκτη **Κεφαλή**. Ξεκινώντας από τον πρώτο κόμβο της λίστας μπορούμε να έχουμε πρόσβαση στη διεύθυνση μνήμης του δεύτερου κόμβου. Από τον δεύτερο κόμβο μπορούμε να έχουμε πρόσβαση στη διεύθυνση μνήμης του τρίτου κόμβου. Και συνεχίζουμε έτσι μέχρι να φτάσουμε στον τέταρτο κόμβο. Αυτός είναι ο μόνος τρόπος για να διασχίσουμε μία συνδεδεμένη λίστα.



Πρόσβαση στους κόμβους μιας συνδεδεμένης λίστας

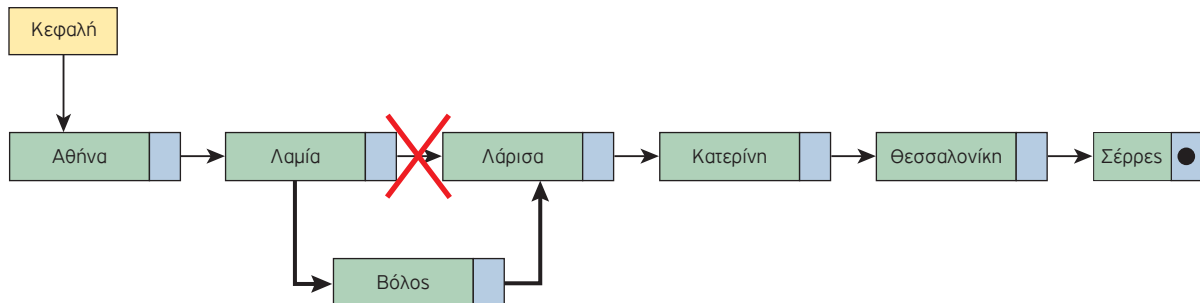
Οι κόμβοι μιας (απλά) συνδεδεμένης λίστας είναι διατεταγμένοι σε μια συγκεκριμένη σειρά, χωρίς αυτό να σημαίνει ότι αποθηκεύονται σε συνεχόμενες θέσεις στη μνήμη. Αντίθετα, είναι διασκορπισμένοι σε όλη τη μνήμη και η σύνδεση μεταξύ τους γίνεται μέσω των δεικτών. Έχουμε άμεση πρόσβαση μόνο στον πρώτο κόμβο της λίστας. Επομένως, για να εντοπίσουμε κάποιον από τους ενδιαμέσους κόμβους, πρέπει να ξεκινήσουμε από τον πρώτο κόμβο της λίστας και να ακολουθήσουμε τους δείκτες με τη σειρά, μέχρι να φτάσουμε στον επιθυμητό κόμβο.

Έστω ότι σχεδιάζουμε στη συνέχεια ένα ταξίδι στην Κεντρική Μακεδονία. Η αφετηρία μας είναι η Αθήνα και το τέρμα του ταξιδιού είναι οι Σέρρες. Έχουμε αποθηκεύσει σε μια συνδεδεμένη λίστα τις πόλεις που είναι πιθανό να επισκεφθούμε (Εικόνα 1.3.4).



Εικόνα 1.3.4. Μια απλά συνδεδεμένη λίστα

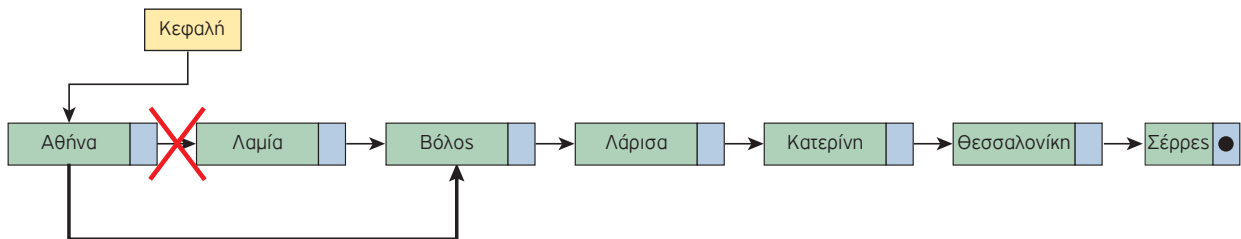
Λίγο πριν το ταξίδι, αποφασίζουμε να επισκεφθούμε τον Βόλο, αμέσως μετά τη Λαμία και πριν φτάσουμε στη Λάρισα. Επομένως, χρειάζεται να εισαγάγουμε ένα νέο κόμβο στη λίστα μας, κάνοντας την κατάλληλη διευθέτηση των δεικτών (Εικόνα 1.3.5).



Εικόνα 1.3.5. Εισαγωγή νέου κόμβου στη λίστα

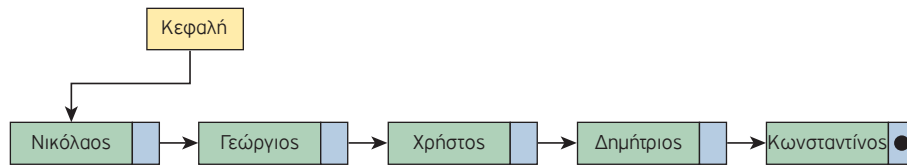
Όπως φαίνεται και στην Εικόνα 1.3.5, οι απαιτούμενες ενέργειες για την εισαγωγή (παρεμβολή) του νέου κόμβου είναι ο δείκτης του δεύτερου κόμβου να δείχνει τον νέο κόμβο και ο δείκτης του νέου κόμβου να δείχνει τον τέταρτο κόμβο (δηλαδή, να πάρει την τιμή που είχε πριν την εισαγωγή ο δείκτης του δεύτερου κόμβου). Με αυτόν τον τρόπο, οι κόμβοι της λίστας διατηρούν τη λογική τους σειρά, αλλά οι φυσικές θέσεις στη μνήμη μπορεί να είναι τελείως διαφορετικές.

Αν, επιπλέον, αποφασίσουμε να μην επισκεφθούμε τη Λαμία, θα πρέπει να διαγράψουμε τον παραπάνω κόμβο από τη λίστα κάνοντας μια εκ νέου διευθέτηση των αντίστοιχων δεικτών (Εικόνα 1.3.6). Έτσι, για τη διαγραφή ενός κόμβου αρκεί ν' αλλάξει τιμή ο δείκτης του προηγούμενου κόμβου και να δείχνει πλέον στον επόμενο αυτού που διαγράφεται, όπως φαίνεται στην Εικόνα 1.3.6. Ο κόμβος που διαγράφηκε (ο δεύτερος) αποτελεί «άχρηστο δεδομένο» και ο χώρος μνήμης που καταλάμβανε, παραχωρείται για άλλη χρήση.



Εικόνα 1.3.6. Διαγραφή κόμβου από τη λίστα

Ας δούμε ένα δεύτερο παράδειγμα με μία συνδεδεμένη λίστα. Ας φανταστούμε έναν αγώνα σκυταλοδρομίας (Εικόνα 1.3.7). Οι αθλητές που λαμβάνουν μέρος είναι τοποθετημένοι με μια συγκεκριμένη σειρά και ο κάθε αθλητής για να ξεκινήσει να τρέχει πρέπει να περιμένει τον προηγούμενο του. Σε περιπτώσεις όπως αυτή, που κάθε στοιχείο συνδέεται με το αμέσως επόμενο, χρησιμοποιούμε τη δομή δεδομένων της συνδεδεμένης λίστας για να αποθηκεύσουμε τις τιμές.

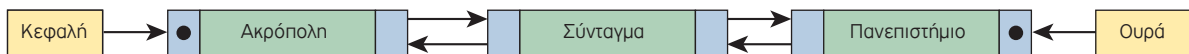


Εικόνα 1.3.7. Το παράδειγμα των αθλητών της σκυταλοδρομίας - Απλά συνδεδεμένη λίστα



Οι συνδεδεμένες λίστες αξιοποιούνται για την υλοποίηση της στοίβας και της ουράς, λόγω της δυνατότητας αυξομείωσης του μεγέθους τους. Μπορείτε να εξηγήσετε γιατί;

Μια λίστα μπορεί να είναι **απλά συνδεδεμένη**, όπως στο παράδειγμα της σκυταλοδρομίας, δηλαδή να μπορούμε να κινηθούμε προς μία μόνο κατεύθυνση, ξεκινώντας από τον πρώτο κόμβο και μετακινούμενοι προς τον τελευταίο, όπως δείχνουν τα βέλη του σχήματος ή να είναι **διπλά συνδεδεμένη (doubly linked list)**, δηλαδή, να μπορούμε να τη διατρέξουμε και προς τις δύο κατευθύνσεις. Η χρήση του δεύτερου δείκτη προσφέρει τη δυνατότητα ξεκινώντας από οποιοδήποτε κόμβο της λίστας να μπορούμε να διαβάσουμε τη λίστα και προς τις δυο κατευθύνσεις. Ένα τέτοιο παράδειγμα είναι οι σταθμοί του μετρό.



Εικόνα 1.3.8. Οι σταθμοί του μετρό - Μια διπλά συνδεδεμένη λίστα

Όπως φαίνεται και στην Εικόνα 1.3.8., κάθε κόμβος της διπλά συνδεδεμένης λίστας, συνδέεται με τον αμέσως επόμενο και τον αμέσως προηγούμενο κόμβο της λίστας. Εκτός, βέβαια, από τον αρχικό και τον τελευταίο κόμβο της λίστας.



Σχεδιάστε οδικές μεταβάσεις

Παρατηρήστε τα σχήματα των απλών και διπλών συνδεδεμένων λιστών. Μπορείτε να εντοπίσετε τις διαφορές τους; Σχεδιάστε στη συνέχεια μία διπλά συνδεδεμένη λίστα που θα αποτυπώνει την οδική μετάβαση μεταξύ Αθηνών και Ιωαννίνων μέσω της νέας Ιονίας οδού.

Σε μια διπλά συνδεδεμένη λίστα διευκολύνεται η ταξινόμηση και η αναζήτηση, ωστόσο, αυξάνεται η πολυπλοκότητα στη διαχείριση των κόμβων, καθώς απαιτείται επιπλέον χώρος για τον δεύτερο δείκτη (επιπρόσθετη μνήμη για κάθε κόμβο).

Διαφορές Λίστας σε σχέση με τον Πίνακα – Πλεονεκτήματα – Μειονεκτήματα

Συνοψίζοντας, θα μπορούσε κάποιος πρόχειρα να συμπεράνει ότι η λίστα δε διαφέρει από τον πίνακα. Παρατηρώντας όμως πιο προσεκτικά, είναι εύκολο να εντοπίσει σημαντικές **διαφορές** μεταξύ τους, οι οποίες παρατίθενται παρακάτω:

- Ο πίνακας θεωρείται μια δομή τυχαίας προσπέλασης, σε αντίθεση με μια λίστα που είναι στην ουσία μια δομή ακολουθιακής ή σειριακής προσπέλασης. Για να φθάσουμε, δηλαδή, σ' έναν κόμβο μιας λίστας πρέπει να περάσουμε από όλους τους προηγούμενους ξεκινώντας από τον πρώτο.
- Ο πίνακας έχει σταθερό μέγεθος, το οποίο δηλώνεται εξ αρχής κατά την υλοποίηση. Αυτό γίνεται, διότι ο πίνακας είναι στατική δομή δεδομένων σε αντίθεση με τη λίστα που είναι δυναμική δομή και το μέγεθός της μπορεί να μεταβάλλεται καθώς εισέρχονται νέοι κόμβοι στη λίστα ή διαγράφονται κάποιοι άλλοι.
- Οι κόμβοι της λίστας αποθηκεύονται σε μη συνεχόμενες θέσεις μνήμης σε αντιδιαστολή με τους πίνακες, όπου τα στοιχεία αποθηκεύονται σε συνεχόμενες θέσεις μνήμης.

Στα **πλεονεκτήματα** των λιστών (έναντι των πινάκων) συγκαταλέγονται τα εξής:

- Το δυναμικό τους μέγεθος,
- η ευκολία εισαγωγής και διαγραφής από οποιοδήποτε μέρος της λίστας, καθώς και
- η μη αναγκαιότητα δήλωσης του μεγέθους τους.

Στα **μειονεκτήματα** των λιστών (έναντι των πινάκων) περιλαμβάνονται τα εξής:

- Η τυχαία πρόσβαση στη λίστα δεν επιτρέπεται. Είναι αδύνατο να φτάσετε στον n -οστό κόμβο μιας απλά συνδεδεμένης λίστας χωρίς πρώτα να περάσετε από όλους τους κόμβους διαδοχικά μέχρι τον συγκεκριμένο κόμβο ξεκινώντας από τον πρώτο κόμβο. Εναλλακτικά, στην περίπτωση της διπλά συνδεδεμένης λίστας μπορείτε να ξεκινήσετε και από τον τελευταίο κόμβο. Επομένως, δεν μπορούμε να πραγματοποιήσουμε με αποτελεσματικό τρόπο δυαδική αναζήτηση σε συνδεδεμένες λίστες.
- Οι συνδεδεμένες λίστες έχουν πολύ μεγαλύτερη επιβάρυνση από τους πίνακες, αφού οι συνδεδεμένοι κόμβοι της λίστας είναι δυναμικά κατανεμημένοι (οι οποίοι είναι λιγότερο αποτελεσματικοί στη χρήση της μνήμης) και κάθε κόμβος στη λίστα πρέπει, επιπλέον, να αποθηκεύσει έναν πρόσθετο δείκτη που θα δείχνει στον επόμενο κόμβο. Στην περίπτωση των διπλά συνδεδεμένων λιστών χρειαζόμαστε επιπλέον έναν δεύτερο δείκτη που θα δείχνει στον προηγούμενο κόμβο.



Διαφορές στην προσπέλαση μεταξύ δομών δεδομένων

Εντοπίστε τις διαφορές στην προσπέλαση όσον αφορά τη λίστα, τη στοίβα και την ουρά. Για την απάντησή σας σκεφθείτε αν η εισαγωγή ενός στοιχείου σε μία στοίβα μπορεί να γίνει σε οποιαδήποτε θέση της.

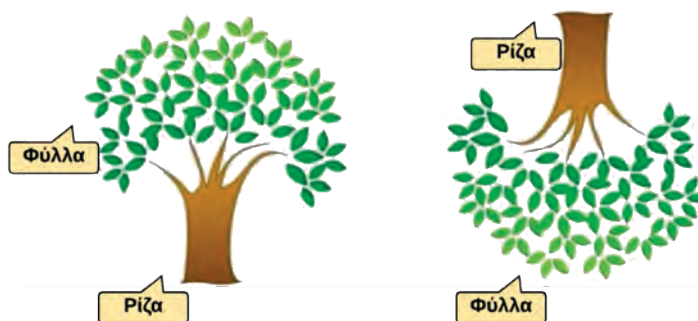
Βασικές πράξεις των συνδεδεμένων λιστών

Οι βασικές πράξεις των συνδεδεμένων λιστών είναι οι παρακάτω:

- Εισαγωγή κόμβου στη λίστα (εισαγωγή κόμβου στην αρχή, στο τέλος της λίστας ή ενδιάμεσα).
- Διαγραφή κόμβου από τη λίστα (διαγραφή από την αρχή, το τέλος της λίστας ή ενδιάμεσα).
- Έλεγχος για το αν η λίστα είναι κενή.
- Αναζήτηση κόμβου για την εύρεση συγκεκριμένου στοιχείου.
- Διάσχιση της λίστας και προσπέλαση των στοιχείων της (π.χ. εκτύπωση των δεδομένων που περιέχονται σε όλους τους κόμβους της λίστας).

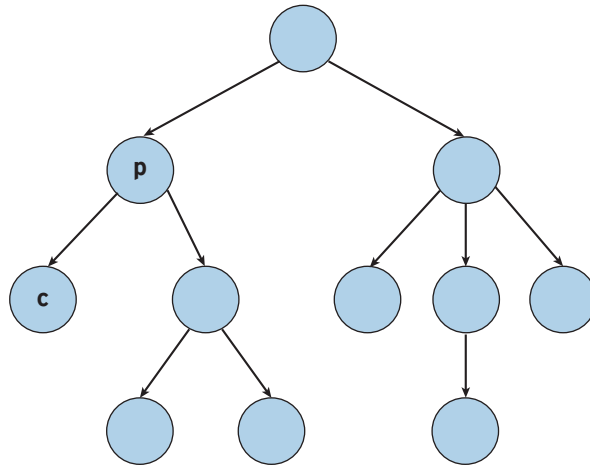
1.3.2 Δένδρα

Μερικές φορές τα πράγματα δεν είναι γραμμικά, όπως έχουμε δει μέχρι τώρα. Σε μία γραμμική δομή, μετά από κάθε στοιχείο ακολουθεί ένα άλλο στοιχείο εκτός και αν είναι το τελευταίο. Τι συμβαίνει όμως στις περιπτώσεις όπου μετά από ένα στοιχείο ακολουθεί όχι ένα, αλλά δύο, τρία ή και περισσότερα στοιχεία; Μπορείτε να φέρετε στον νου σας εικόνες, όπου εμφανίζεται αυτή η συγκεκριμένη συνδεσμολογία των στοιχείων; Μήπως μία από αυτές τις εικόνες είναι το δένδρο; Παρατηρείστε ένα δένδρο στη φύση. Έχει ρίζα και φύλλα, όπως φαίνεται στο αριστερό μέρος της Εικόνας 1.3.9. Η ρίζα είναι στο κάτω μέρος και τα φύλλα στο πάνω μέρος. Στην επιστήμη της Πληροφορικής, όμως, βλέπουμε τα δένδρα ανάποδα. Τα φύλλα είναι στο κάτω μέρος και η ρίζα στο επάνω μέρος, όπως φαίνεται στο δεξιό μέρος της Εικόνας 1.3.9.



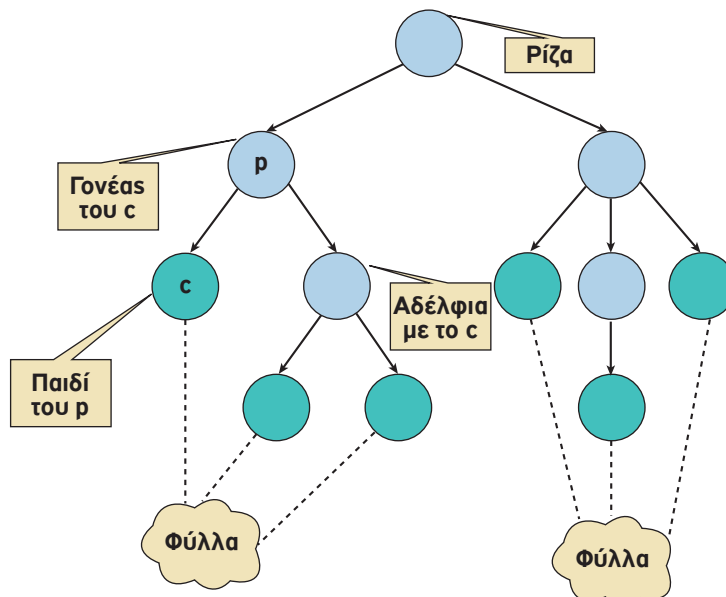
Εικόνα 1.3.9. Τα δένδρα στη φύση και στην Πληροφορική

Ας ξανασχεδιάσουμε το δένδρο που βρίσκεται στο δεξιό μέρος της Εικόνας 1.3.9 με έναν πιο αφαιρετικό τρόπο και στη συνέχεια ας προσπαθήσουμε να ορίσουμε αυτό το οποίο βλέπουμε σε ένα δένδρο, προσδιορίζοντας τη δομή του. Έτσι λοιπόν, θα λέγαμε ότι ένα **δένδρο** αποτελείται από κόμβους, οι οποίοι συνδέονται μεταξύ τους με ακμές. Στην Εικόνα 1.3.10 βλέπουμε τους κόμβους με γαλάζιο χρώμα και τις ακμές ως βέλη μεταξύ των κόμβων.



Εικόνα 1.3.10. Δομή ενός δένδρου

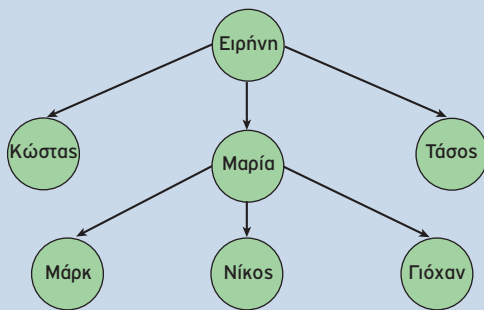
Όταν δύο κόμβοι συνδέονται μεταξύ τους με μία ακμή, τότε ονομάζουμε «γονέα» τον κόμβο από τον οποίο ξεκινάει η ακμή και «παιδί» τον κόμβο στον οποίο καταλήγει η ακμή. Στην Εικόνα 1.3.11 ο κόμβος p είναι γονέας του κόμβου c και ο κόμβος c είναι παιδί του κόμβου p . Ένας κόμβος μπορεί να έχει κανένα, ένα ή περισσότερα παιδιά. Όλοι οι κόμβοι, εκτός από έναν, έχουν ακριβώς έναν γονέα. Ο κόμβος χωρίς γονέα ονομάζεται «ρίζα» (root) και βρίσκεται στην κορυφή του δένδρου. Κόμβοι με τον ίδιο γονέα ονομάζονται «αδέλφια». Οι κόμβοι χωρίς παιδιά ονομάζονται «φύλλα».



Εικόνα 1.3.11. Σχέσεις μεταξύ των κόμβων ενός δένδρου



Οικογενειακές Σχέσεις



Εικόνα 1.3.12. Οικογενειακό δένδρο

Στο δένδρο της Εικόνας 1.3.12 βρείτε ποιος/ποιοι κόμβος/κόμβοι είναι:

- ο γονέας του Τάσου
- τα φύλλα του δένδρου
- η ρίζα του δένδρου
- τα παιδιά της Μαρίας
- ποια είναι αδέλφια

Μπορούμε να έχουμε ένα απλό δένδρο, το οποίο να απαρτίζεται από έναν μόνο κόμβο. Αυτός ο κόμβος είναι **και ρίζα** του απλού αυτού δένδρου, διότι δεν έχει γονέα **και φύλλο**, και διότι δεν έχει παιδιά.

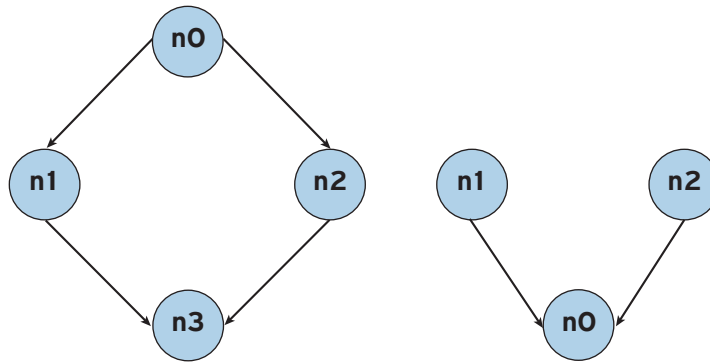


Ένα **δένδρο (tree)** είναι μία δομή που αποτελείται από ένα σύνολο κόμβων και ένα σύνολο ακμών μεταξύ των κόμβων με βάση τους εξής κανόνες:

- Υπάρχει ένας ξεχωριστός κόμβος που ονομάζεται ρίζα. Αυτός είναι ένας κόμβος χωρίς γονέα.
- Για κάθε κόμβο c , εκτός από τη ρίζα, υπάρχει μόνο μια ακμή που καταλήγει στον κόμβο αυτόν ξεκινώντας από κάποιον άλλον κόμβο p . Ο κόμβος p ονομάζεται γονέας του c και ο κόμβος c παιδί του p .
- Για κάθε κόμβο υπάρχει μία μοναδική διαδρομή, δηλαδή, μια ακολουθία διαδοχικών ακμών, που ξεκινάει από τη ρίζα και τερματίζει σε αυτόν τον κόμβο.

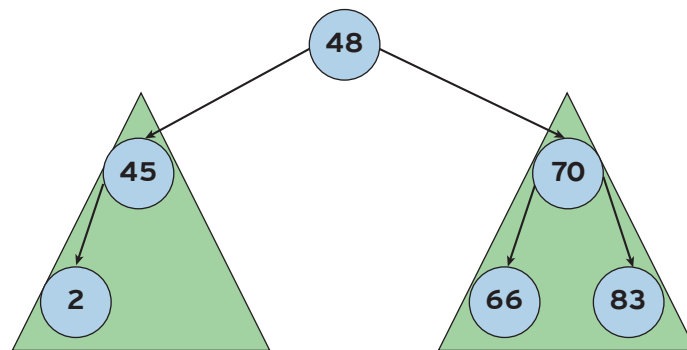
Δένδρο θεωρούμε και το κενό δένδρο, δηλαδή το δένδρο που δεν έχει ούτε κόμβους, ούτε ακμές. Το κενό δένδρο είναι το μόνο δένδρο χωρίς ρίζα.

Στην Εικόνα 1.3.13 παρουσιάζουμε παραδείγματα δομών που δεν είναι δένδρα. Η δομή στα αριστερά δεν είναι δένδρο επειδή ο κόμβος $n3$ έχει δύο γονείς, τους $n1$ και $n2$ και, όπως ξέρουμε, σε ένα δένδρο ένας κόμβος πρέπει να έχει ακριβώς έναν γονέα, με εξαίρεση τη ρίζα, που δεν έχει κανέναν. Επίσης, υπάρχουν δύο διαδρομές από την ρίζα $n0$ προς τον κόμβο $n3$, πράγμα που και αυτό απαγορεύεται σε ένα δένδρο. Η δομή στα δεξιά δεν είναι δένδρο διότι υπάρχουν δύο κόμβοι χωρίς γονέα, οι $n1$ και $n2$. Είναι σαν να λέμε ότι έχουμε δύο ρίζες στο δένδρο αυτό. Γνωρίζουμε όμως ότι η ρίζα είναι μοναδική σε ένα δένδρο.



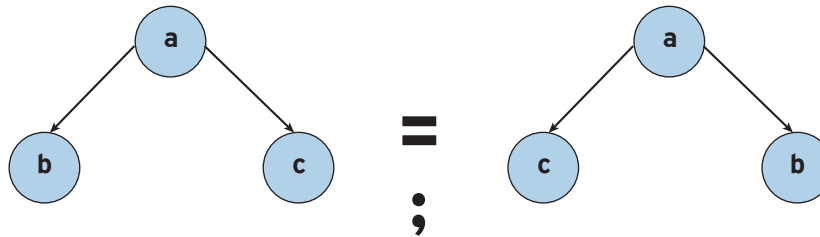
Εικόνα 1.3.13. Δομές που δεν είναι δένδρα

Μέσα σε ένα δένδρο μπορούμε να εντοπίσουμε και άλλα μικρότερα δένδρα, που ονομάζονται υποδένδρα. Πιο συγκεκριμένα, κάθε κόμβος ενός δένδρου μπορεί να θεωρηθεί ως ρίζα ενός υποδένδρου, δηλαδή ενός άλλου μικρότερου δένδρου, που ξεκινάει από τον κόμβο αυτόν. Στο δένδρο της Εικόνας 1.3.14, όπως φαίνεται και από το σχήμα, ο κόμβος 48 είναι ρίζα και έχει δύο υποδένδρα που ξεκινούν από τους κόμβους 45 και 70 αντίστοιχα. Ο κόμβος 45 έχει ένα υποδένδρο που αποτελείται από τον κόμβο 2. Ο κόμβος 70 έχει δύο υποδένδρα που αποτελούνται από τους κόμβους 66 και 83 αντίστοιχα. Τα υποδένδρα των κόμβων 2, 66 και 83 είναι κενά.



Εικόνα 1.3.14. Αριστερό και δεξιό υποδένδρο του κόμβου 48

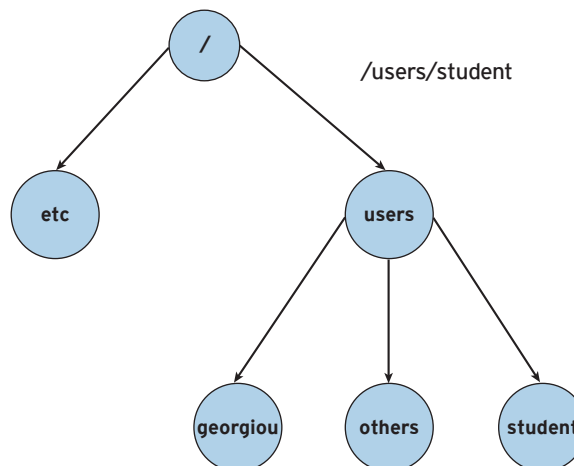
Τα δύο δένδρα της Εικόνας 1.3.15 είναι ίδια ή όχι; Μιλήσαμε για τη σχέση γονέα-παιδιού, αλλά τι γίνεται με τη σχέση μεταξύ των αδελφών; Έχει σημασία η σειρά των αδελφών b και c; Όχι πάντοτε. Για παράδειγμα, αν θέλουμε να μοντελοποιήσουμε την ιεραρχική σχέση των μελών μιας οικογένειας και μας ενδιαφέρει να οργανώσουμε τα αδέλφια σύμφωνα με την ηλικία τους, τότε τα αδέλφια που θα έχουν γεννηθεί νωρίτερα θα τοποθετηθούν στην δενδρική δομή πιο αριστερά σε σχέση με αυτά που θα έχουν γεννηθεί αργότερα. Σε αυτή την περίπτωση, που για κάθε κόμβο υπάρχει μία γραμμική σχέση μεταξύ των παιδιών του κόμβου αυτού, αναφερόμαστε σε ένα διατεταγμένο δένδρο.



Εικόνα 1.3.15. Πότε δύο δένδρα είναι ίδια

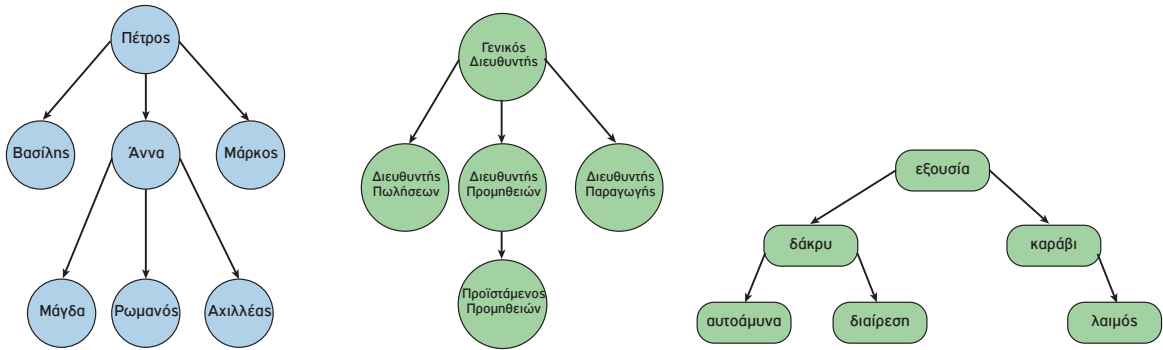
Τα δένδρα είναι μία μη-γραμμική ευέλικτη δομή δεδομένων που χρησιμοποιούνται σε πολλούς τομείς της επιστήμης των υπολογιστών, συμπεριλαμβανομένων των λειτουργικών συστημάτων, των γραφικών, των συστημάτων βάσεων δεδομένων, των παιχνιδιών, της τεχνητής νοημοσύνης και της δικτύωσης υπολογιστών.

Έχετε σκεφτεί γιατί υπάρχουν όμως τόσα δένδρα; Υπάρχουν δύο λόγοι για τους οποίους τα δένδρα είναι τόσο ισχυρά. Ο πρώτος λόγος αναφέρεται στη δυναμικότητα των δένδρων. Είναι πολύ εύκολο να προσθέσετε, να αφαιρέσετε ή να αναζητήσετε ένα στοιχείο σε ένα δένδρο, όπως θα δούμε στη συνέχεια. Ο δεύτερος βασικός λόγος είναι ότι η δομή των δένδρων μεταφέρει πληροφορίες. Για παράδειγμα, ας θεωρήσουμε το σύστημα αρχείων του υπολογιστή, όπως φαίνεται στην Εικόνα 1.3.16. Είναι πολύ εύκολο να προσθέσετε ένα νέο κατάλογο για τον καθηγητή "georgiou". Επίσης, λόγω του ότι ο κατάλογος "users" είναι παιδί της ρίζας "/" και ότι ο κατάλογος "student" είναι παιδί του "users" μπορούμε να συμπεράνουμε ότι υπάρχει η διαδρομή "/users/student".



Εικόνα 1.3.16. Μη-γραμμική ευέλικτη δομή δεδομένων

Συνήθως, όταν αναφερόμαστε στα δένδρα, μας έρχονται στο νου διάφορες αναπαραστάσεις δεδομένων του πραγματικού κόσμου αλλά και της Πληροφορικής που διέπονται από ένα είδος φυσικής ιεραρχίας, όπως είναι το οικογενειακό δένδρο (Εικόνα 1.3.17.α), η δομή ενός οργανισμού ή μιας εταιρείας (Εικόνα 1.3.17.β), ο πίνακας περιεχομένων ενός βιβλίου, τα αρχεία και οι φάκελοι ενός υπολογιστή, ένα λεξικό (Εικόνα 1.3.17.γ) (περισσότερα στην ενότητα των δυαδικών δένδρων αναζήτησης), τα μέρη που απαρτίζουν την μηχανή ενός αυτοκινήτου, τα συστατικά μιας πρότασης (Εικόνα 1.3.18) κ.ά.



Εικόνα 1.3.17.

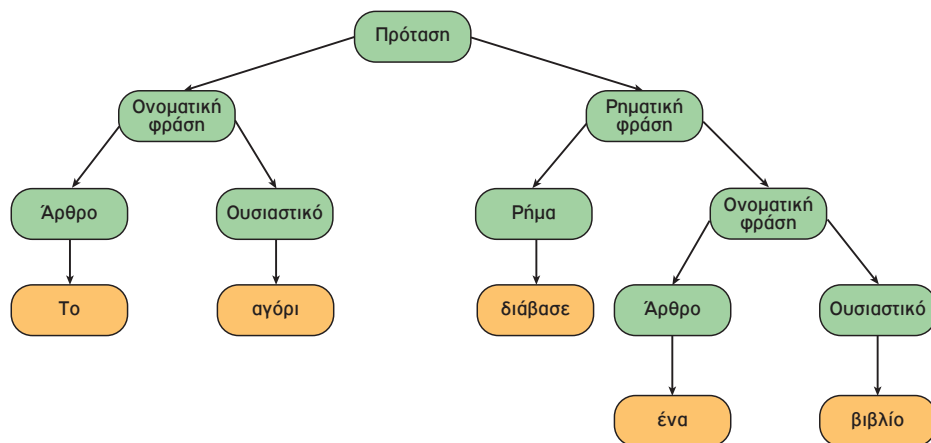
α. Οικογενειακό δένδρο

β. Οργανόγραμμα μιας εταιρείας

γ. Οργάνωση ενός λεξικού

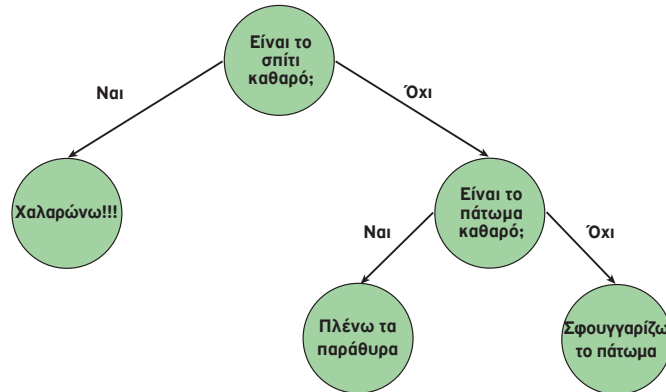
Έχετε σκεφτεί, όμως, ότι τα δένδρα, πέρα από μια αποτελεσματική οργάνωση και διαχείριση δεδομένων, αποτελούν τη βάση αρκετών αλγορίθμων επίλυσης προβλήματος, όπως είναι για παράδειγμα η συμπίεση εικόνων, η ταξινόμηση, η αυτόματη συμπλήρωση λέξεων σε συσκευές κινητών τηλεφώνων, η μεταγλώττιση ενός προγράμματος και η λήψη αποφάσεων;

Για παράδειγμα, ένα απλό πρόβλημα λήψης απόφασης μπορεί να είναι αυτό της καθαριότητας του φοιτητικού σπιτιού σας. Κάποια στιγμή θα κληθείτε να πάρετε αποφάσεις λαμβάνοντας υπόψη κάποιους παράγοντες. Μήπως θα σας βοηθούσε ένα δένδρο απόφασης παρόμοιο με αυτό που εμφανίζεται στην Εικόνα 1.3.19; Ειδικότερα, η μεθοδολογία των δένδρων αποφάσεων έχει γίνει ιδιαίτερα δημοφιλής σε περιπτώσεις ιατρικών διαγνώσεων.



Εικόνα 1.3.18. Η δομή μιας πρότασης

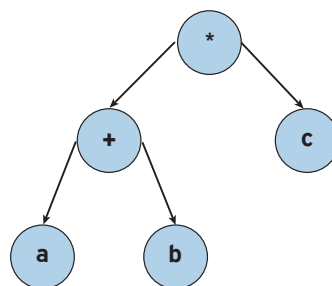
Τα δένδρα απόφασης, όπως πολύ εύκολα μπορείτε να συμπεράνετε από την Εικόνα 1.3.19, είναι δένδρα στα οποία κάθε κόμβος αντιπροσωπεύει ένα χαρακτηριστικό (ιδιότητα), κάθε ακμή αντιπροσωπεύει μια απόφαση (κανόνα) και κάθε φύλλο αντιπροσωπεύει ένα αποτέλεσμα. Στους αλγορίθμους μηχανικής μάθησης (machine learning) τα δένδρα απόφασης έχουν πρωτεύοντα ρόλο.



Εικόνα 1.3.19. Δένδρο Απόφασης

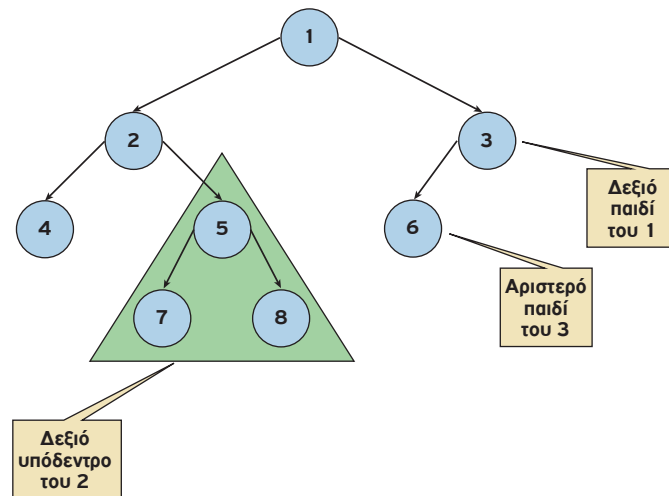
Από την άλλη πλευρά, γνωρίζετε ότι όταν παίζετε παιχνίδια στον υπολογιστή, όπως είναι το σκάκι, η τρίλιζα, το τάβλι και πολλά άλλα, ο υπολογιστής χρησιμοποιεί ένα ειδικό δένδρο, που ονομάζεται δένδρο του παιχνιδιού (game tree), το οποίο μοντελοποιεί όλες τις πιθανές κινήσεις των παικτών για να σας νικήσει; Θα μπορούσε κάθε κόμβος στο δένδρο, που αντιπροσωπεύει μία συγκεκριμένη κατάσταση παιχνιδιού, να περιέχει πληροφορίες σχετικά με το ποιος παίκτης έχει τη μεγαλύτερη πιθανότητα να κερδίσει από οποιαδήποτε πιθανή κίνηση;

Διαδεδομένα είναι επίσης τα δένδρα για την αναπαράσταση και κατ' επέκταση τον υπολογισμό αριθμητικών εκφράσεων, όπως αυτό της Εικόνας 1.3.20.

Εικόνα 1.3.20. $(a+b)*c$

Διαδικά Δένδρα

Ένα διαδικό δένδρο (binary tree) είναι ένα διατεταγμένο δένδρο, στο οποίο κάθε κόμβος έχει το πολύ δύο παιδιά, το αριστερό και το δεξί παιδί. Μπορούμε, συνεπώς, να μιλάμε για αριστερό και δεξιό υποδένδρο ενός κόμβου. Στο διαδικό δένδρο της Εικόνας 1.3.21, ο κόμβος 3 έχει ως αριστερό υποδένδρο, το δένδρο με μοναδικό κόμβο το 6 και ως δεξιό υποδένδρο, το κενό δένδρο. Προφανώς, αν ανταλλάξουμε το αριστερό με το δεξιό υποδένδρο ενός κόμβου παίρνουμε ένα διαφορετικό δένδρο.



Εικόνα 1.3.21. Διαδικό δένδρο



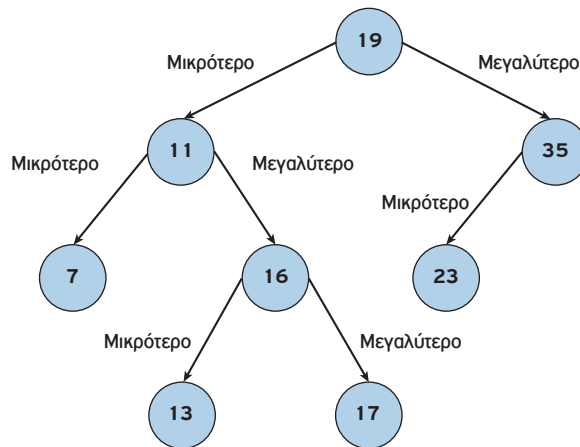
Στην Εικόνα 1.3.21 μπορείτε να βρείτε ποιοι κόμβοι έχουν ένα, δύο ή κανένα παιδί;

Διαδικά Δένδρα Αναζήτησης

Γνωρίζετε ότι οι αλγόριθμοι της αναζήτησης, για παράδειγμα όλων των email που έχετε λάβει ή στείλει σε μία συγκεκριμένη περίοδο ή της εύρεσης όλων των λέξεων, που αρχίζουν από την συμβολοσειρά «πληρ», αξιοποιούν μία ειδική κατηγορία διαδικών δένδρων, αυτών των διαδικών δένδρων αναζήτησης;

Θα χρησιμοποιήσουμε τώρα τα διαδικά δένδρα με έναν συγκεκριμένο τρόπο. Θα αποθηκεύσουμε δεδομένα με έναν τρόπο που θα μας επιτρέψει να τα βρίσκουμε πιο εύκολα. Θα διαπιστώσετε και μόνοι σας στη συνέχεια ότι η ιδέα πίσω από ένα διαδικό δένδρο αναζήτησης είναι παρόμοια με αυτήν της διαδικής αναζήτησης σε έναν ταξινομημένο πίνακα.

Ένα διαδικό δένδρο αναζήτησης (binary search tree) είναι ένα διαδικό δένδρο, όπου για κάθε κόμβο u , όλοι οι κόμβοι του αριστερού υποδένδρου έχουν τιμές μικρότερες της τιμής του κόμβου u και όλοι οι κόμβοι του δεξιού υποδένδρου έχουν τιμές μεγαλύτερες (ή ίσες) της τιμής του κόμβου u . Για λόγους απλούστευσης θεωρούμε ότι δεν υπάρχουν τιμές ίσες με την τιμή του κόμβου u . Στην Εικόνα 1.3.22 παρουσιάζεται το παράδειγμα ενός διαδικού δένδρου αναζήτησης.



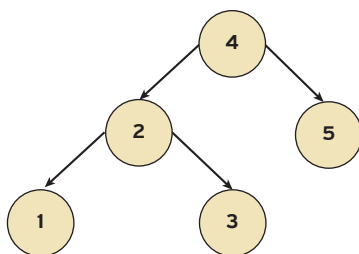
Εικόνα 1.3.22. Δυαδικό δένδρο αναζήτησης

Το δυαδικό δένδρο της Εικόνας 1.3.23.α πληροί τα κριτήρια του δυαδικού δένδρου αναζήτησης, σε αντίθεση με το δυαδικό δένδρο της Εικόνας 1.3.23.β.

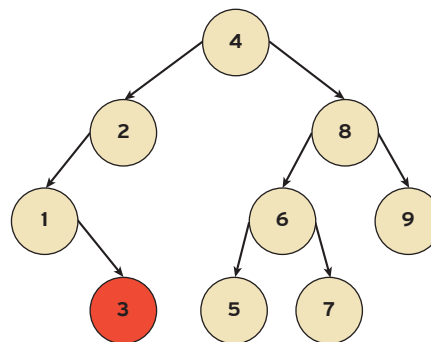
Στο δυαδικό δένδρο, της Εικόνας 1.3.23.β, παρατηρούμε τα εξής, όσον αφορά τον κόμβο 3:

- Ας θεωρήσουμε το υποδένδρο με ρίζα το 1. Ο κόμβος 3 βρίσκεται στο δεξιό υποδένδρο αυτού και είναι μεγαλύτερος από τον κόμβο/ρίζα 1.
- Ας θεωρήσουμε το υποδένδρο με ρίζα το 2. Ο κόμβος 3 βρίσκεται στο αριστερό υποδένδρο αυτού αλλά δεν είναι μικρότερος από τον κόμβο/ρίζα 2.

Εξαιτίας του τελευταίου γεγονότος, δηλαδή της ύπαρξης ενός τουλάχιστον κόμβου (στην περίπτωσή μας του κόμβου 3) που δεν πληροί το κριτήριο του δυαδικού δένδρου αναζήτησης, συμπεραίνουμε ότι δεν μπορούμε να κατατάξουμε το δυαδικό αυτό δένδρο στα δυαδικά δένδρα αναζήτησης.



Εικόνα 1.3.23. α: Δυαδικό δένδρο αναζήτησης



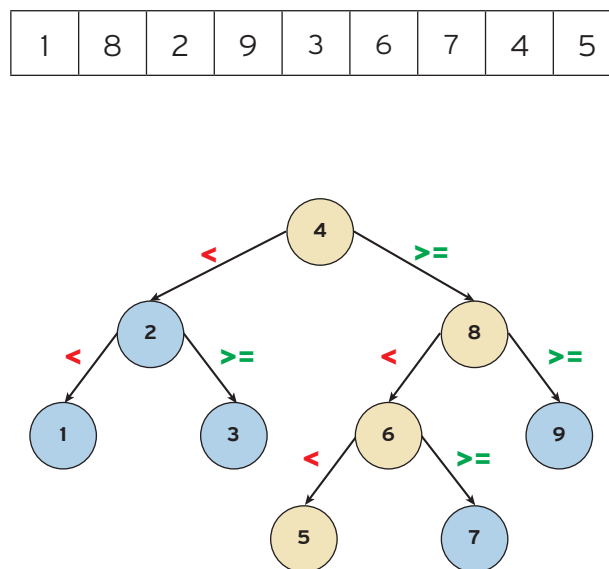
β: Μη δυαδικό δένδρο αναζήτησης



Στην Εικόνα 1.3.23.β, σε ποιο σημείο του δένδρου θα μπορούσε να τοποθετηθεί ο κόμβος 3, έτσι ώστε το δένδρο αυτό να είναι ένα δυαδικό δένδρο αναζήτησης;

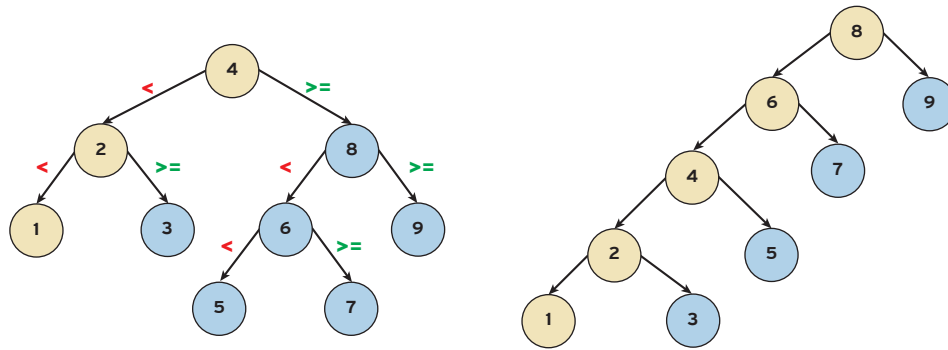
Ποιο νομίζετε ότι είναι το πλεονέκτημα των δυαδικών δένδρων αναζήτησης; Το πλεονέκτημα βρίσκεται στο ίδιο το όνομα και συγκεκριμένα στη λέξη «αναζήτηση». Η αναζήτηση για μια συγκεκριμένη τιμή γίνεται ταχύτερα χάρη στον τρόπο αποθήκευσης των τιμών. Για παράδειγμα, ποια είναι η χειρότερη περίπτωση που μπορούμε να συναντήσουμε όταν ψάχνουμε μία τιμή σε μια γραμμική δομή, όπως είναι ο πίνακας; Θεωρείστε ότι ψάχνετε το στοιχείο 5 στον πίνακα της Εικόνας 1.3.24. Στην περίπτωση αυτή, θα χρειαζόταν να διασχίσουμε ολόκληρο τον πίνακα για να βρούμε το στοιχείο 5. Τώρα, θεωρείστε το δένδρο της Εικόνας 1.3.24. Μετά από πόσες συγκρίσεις θα βρείτε το στοιχείο 5;

Ας δούμε τον αλγόριθμο της αναζήτησης σε ένα δυαδικό δένδρο αναζήτησης μέσα από μερικά παραδείγματα. Αρχικά, συγκρίνετε τη ρίζα του δένδρου (δηλαδή το 4) με το υπό αναζήτηση στοιχείο (δηλαδή το 5). Ο αριθμός που αναζητάτε είναι μεγαλύτερος από τη ρίζα. Συνεπώς συνεχίζετε με το δεξιό υποδένδρο και αγνοείτε το αριστερό υποδένδρο. Στη συνέχεια, συγκρίνετε το 8 με το 5. Επειδή το 8 είναι μεγαλύτερο από το 5, συνεχίζετε με το αριστερό υποδένδρο. Κατόπιν, συναντάτε το 6 που είναι και αυτό μεγαλύτερο από το 5. Μεταβαίνετε, λοιπόν, στο αριστερό υποδένδρο του 6, όπου βρίσκετε το στοιχείο 5 που αναζητάτε. Στην περίπτωση που δεν υπάρχει το στοιχείο στο δένδρο (για παράδειγμα αν ψάχναμε το στοιχείο 12), η αναζήτηση καταλήγει σε ένα κενό υποδένδρο και εκεί τερματίζει η όλη διαδικασία. Με βάση τον αλγόριθμο αυτό, μειώνουμε δραματικά τον χρόνο για να βρούμε το στοιχείο που ψάχνουμε και αυτό διότι περιορίζουμε αισθητά τους κόμβους τους οποίους επισκεπτόμαστε. Κάθε φορά αφήνουμε στην άκρη ένα υποδένδρο και συνεχίζουμε με το άλλο.



Εικόνα 1.3.24. Αλγόριθμος αναζήτησης σε ένα δυαδικό δένδρο αναζήτησης

Θεωρείστε ότι αναζητούμε το στοιχείο 1 στα δυαδικά δένδρα αναζήτησης της Εικόνας 1.3.25. Σε ποιο από τα δύο δυαδικά δένδρα αναζήτησης θα βρούμε πιο γρήγορα την τιμή 1; Μήπως στο πρώτο, διότι σε κάθε βήμα απορρίπτουμε όσο περισσότερους κόμβους είναι δυνατόν; Οπτικά, θα μπορούσαμε να θεωρήσουμε ότι το πρώτο δένδρο είναι πιο «ισορροπημένο» σε σχέση με το δεύτερο; Παρατηρούμε, λοιπόν, χωρίς να αναφερθούμε διεξοδικά στα ισορροπημένα δένδρα, ότι αν θέλουμε να έχουμε γρήγορους αλγόριθμους αναζήτησης πρέπει να αποθηκεύουμε τις τιμές στα δυαδικά δένδρα αναζήτησης με έναν συγκεκριμένο τρόπο.

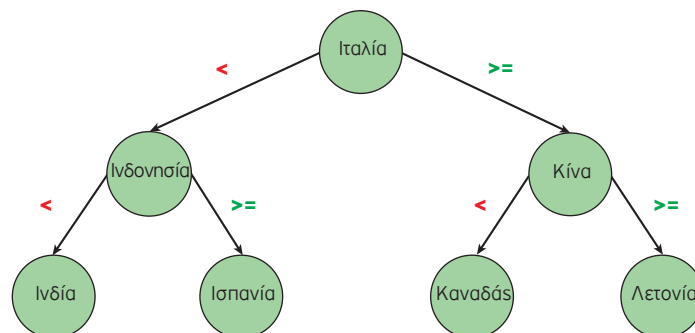


Εικόνα 1.3.25. Η σημασία της δομής ενός δυαδικού δένδρου αναζήτησης στην εύρεση στοιχείων

Σας θυμίζει ο αλγόριθμος αυτός τον αλγόριθμο της δυαδικής αναζήτησης που έχετε συναντήσει στους ταξινομημένους πίνακες; Ο αλγόριθμος της δυαδικής αναζήτησης «τρέχει» γρήγορα στους ταξινομημένους πίνακες σε αντίθεση με τους αλγορίθμους της εισαγωγής και της διαγραφής στοιχείου που είναι πιο χρονοβόροι και αυτό διότι πρέπει να διατηρηθεί η υπάρχουσα διάταξη των στοιχείων. Ας μην ξεχνάμε επίσης ότι οι πίνακες είναι στατικές δομές και επομένως, δεν έχουν δυνατότητα αυξομείωσης του μεγέθους τους. Αυτό σημαίνει ότι κάθε επόμενη εισαγωγή στοιχείου θα οδηγήσει ενδεχομένως σε αντιγραφή όλων των στοιχείων του πίνακα σε έναν μεγαλύτερο πίνακα.

Τα δυαδικά δένδρα αναζήτησης συνδυάζουν τα πλεονεκτήματα των λιστών, όσον αφορά τις πράξεις της εισαγωγής και της διαγραφής, αλλά και τα πλεονεκτήματα των ταξινομημένων πινάκων, όσον αφορά την πράξη της αναζήτησης. Φανταστείτε ότι μετατρέπετε έναν ταξινομημένο πίνακα σε ένα δυαδικό δένδρο, όπως παρουσιάζεται στην Εικόνα 1.3.26. Αυτό μας δίνει τη δυνατότητα να αναζητήσουμε ένα στοιχείο το ίδιο γρήγορα όσο και σε έναν ταξινομημένο πίνακα αλλά και να εισαγάγουμε και να διαγράψουμε εύκολα ένα στοιχείο ακριβώς επειδή δουλεύουμε με δένδρα και όχι με πίνακες.

Ινδία	Ινδονησία	Ισπανία	Ιταλία	Καναδάς	Κίνα	Λετονία
-------	-----------	---------	--------	---------	------	---------



Εικόνα 1.3.26. Αναζήτηση σε ταξινομημένο πίνακα και σε δυαδικό δένδρο αναζήτησης

1.3.3 Γράφοι

Τώρα που μελετήσαμε τις δενδρικές δομές δεδομένων, προκύπτει το ερώτημα «από πού προέρχονται τα δένδρα;». Τα δένδρα είναι στην πραγματικότητα ένα υποσύνολο των γράφων. Αλλά προκειμένου να καταλάβετε πραγματικά γιατί χρησιμοποιούμε γράφους και τι είναι αυτοί, θα πρέπει να αναφερθούμε στη θεωρία γράφων.

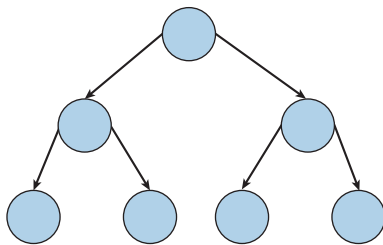
Είδαμε ότι το πιο θεμελιώδες χαρακτηριστικό των μη γραμμικών δομών είναι ότι τα δεδομένα τους δεν ακολουθούν μια σειρά - όπως στους πίνακες ή τις συνδεδεμένες λίστες. Τα δένδρα, όπως είδαμε, ξεκινούν με έναν κόμβο ρίζας και μπορεί να συνδεθούν με άλλους κόμβους, κάτι που σημαίνει ότι θα μπορούσαν να περιέχουν δευτερεύοντα δένδρα στο εσωτερικό τους. Τα δένδρα, γενικά, διέπονται από συγκεκριμένους κανόνες ενώ σε ορισμένους τύπους δένδρων ισχύουν ιδιαίτεροι κανόνες, όπως στα δυαδικά δένδρα αναζήτησης, στα οποία οι κόμβοι μπορεί να έχουν μόνο δύο συνδέσεις με δύο κόμβους ανά πάσα στιγμή.

Αλλά τι θα γίνει αν αγνοήσουμε αυτούς τους κανόνες; Τότε, δεν αναφερόμαστε σε δένδρα αλλά σε μία νέα δυναμική δομή δεδομένων, που ονομάζεται γράφος. Τα δένδρα δεν είναι παρά περιορισμένοι τύποι γράφων. Ένα δένδρο θα είναι πάντα ένα γράφος, αλλά δεν είναι όλοι οι γράφοι δένδρα.

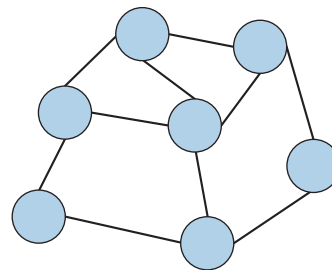
Τι είναι αυτό που κάνει ένα δένδρο διαφορετικό από έναν γράφο;

Ένα δένδρο μπορεί μόνο να ρέει προς μία κατεύθυνση, από τον κόμβο ρίζας σε κόμβους φύλλων ή κόμβους παιδιών. Ένα δένδρο μπορεί να έχει μόνο μονόδρομες συνδέσεις - ένας κόμβος παιδιού μπορεί να έχει μόνο έναν γονέα και ένα δένδρο δεν μπορεί να έχει βρόχους ή κυκλικούς δεσμούς (Εικόνα 1.3.27.α).

Με τους γράφους, όλοι αυτοί οι περιορισμοί δεν υπάρχουν. Οι γράφοι δεν έχουν την έννοια ενός κόμβου «ρίζας». Οι κόμβοι μπορούν να συνδεθούν με οποιονδήποτε τρόπο. Για παράδειγμα, ένας κόμβος μπορεί να συνδεθεί με άλλους πέντε (Εικόνα 1.3.27.β)! Οι γράφοι, επίσης, δεν έχουν «μονοκατευθυντική» ροή - αντ' αυτού, μπορεί να έχουν κατεύθυνση ή να μην έχουν καμιά κατεύθυνση.



α: Δένδρο



β: Γράφος

Εικόνα 1.3.27.



Ένας **γράφος (graph)** είναι μία δομή που αποτελείται από ένα σύνολο κόμβων (ή σημείων ή κορυφών) και ένα σύνολο γραμμών (ή ακμών ή τόξων) που ενώνουν μερικούς ή όλους τους κόμβους. Ο γράφος αποτελεί την πιο γενική δομή δεδομένων, με την έννοια ότι όλες οι προηγούμενες δομές που παρουσιάστηκαν μπορούν να θεωρηθούν περιπτώσεις γράφων.

Τύποι Γράφων

As εξετάσουμε τους δύο τύπους γράφων που είναι αρκετά εύκολο να εντοπιστούν και είναι αρκετά συνηθισμένοι στα προβλήματα θεωρίας γράφων: οι **κατευθυνόμενοι** γράφοι και οι **μη κατευθυνόμενοι** γράφοι.

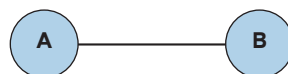
Όπως έχουμε αναφέρει, σε έναν γράφο δεν υπάρχουν πραγματικοί κανόνες για τον τρόπο που ένας κόμβος συνδέεται με έναν άλλο κόμβο. Οι ακμές μπορούν να συνδέσουν τους κόμβους με οποιονδήποτε τρόπο. Οι διαφορετικοί τύποι ακμών είναι πολύ σημαντικοί όταν πρόκειται για την αναγνώριση και τον καθορισμό του τύπου των γράφων. Ως επί το πλείστον, οι γράφοι μπορούν να έχουν δύο τύπους ακμών: ακμές που έχουν κατεύθυνση (κατευθυνόμενες ακμές) και ακμές που δεν έχουν κατεύθυνση (μη κατευθυνόμενες ακμές). Σε μια κατευθυνόμενη ακμή, δύο κόμβοι συνδέονται με πολύ συγκεκριμένο τρόπο.

Στην εικόνα 1.3.28.α, όπου ο κόμβος A συνδέεται με τον κόμβο B, υπάρχει ένας μόνο τρόπος να ταξιδέψουμε μεταξύ αυτών των δύο κόμβων και αυτός είναι από τον κόμβο A στον κόμβο B. Είναι πολύ συνηθισμένο να αναφέρουμε τον κόμβο από τον οποίο ξεκινάμε ως προέλευση και τον κόμβο στον οποίο ταξιδεύουμε ως προορισμό. Σε μια κατευθυνόμενη ακμή, μπορούμε να ταξιδέψουμε μόνο από την προέλευση στον προορισμό, και ποτέ το αντίστροφο (Εικόνα 1.3.28.α). Ωστόσο, σε μια μη κατευθυνόμενη ακμή, η διαδρομή μεταξύ των δύο κόμβων είναι αμφίδρομη, πράγμα που σημαίνει ότι οι κόμβοι προέλευσης και προορισμού δεν είναι σταθεροί.

Αυτή η διαφοροποίηση είναι πραγματικά πολύ σημαντική, επειδή οι ακμές σε έναν γράφο καθορίζουν το πώς αποκαλείται ο γράφος.



α: Κατευθυνόμενη ακμή



β: μη κατευθυνόμενη ακμή

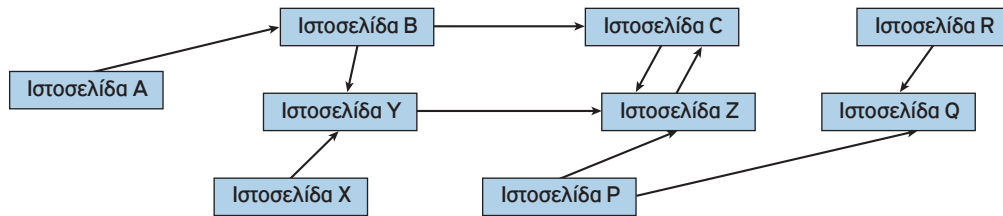
Εικόνα 1.3.28.



Εάν όλες οι ακμές σε έναν γράφο έχουν κατεύθυνση, ο γράφος ονομάζεται **κατευθυνόμενος γράφος (directed graph)**.

Εάν όλες οι ακμές σε έναν γράφο δεν έχουν κατεύθυνση, ο γράφος ονομάζεται **μη κατευθυνόμενος γράφος (undirected graph)**.

Οι γράφοι είναι γύρω μας, απλά δεν τους βλέπουμε. Ο Παγκόσμιος Ιστός (WWW) είναι ένας τεράστιος γράφος (Εικόνα 1.3.29)! Όταν κάνουμε κλικ ανάμεσα σε ιστότοπους και πλοηγούμαστε μεταξύ των διευθύνσεων URL, κάνουμε πραγματικά περιήγηση σε έναν γράφο. Μερικές φορές αυτές οι αναπαραστάσεις γράφων έχουν κόμβους με ακμές που είναι μη κατευθυνόμενες - μπορούμε να προχωρήσουμε από μια ιστοσελίδα σε μια άλλη και το αντίστροφο - και άλλες ακμές που κατευθύνονται - μπορούμε να πάμε μόνο από την ιστοσελίδα A στην **ιστοσελίδα B, και ποτέ το αντίστροφο**.



Εικόνα 1.3.29. Παγκόσμιος Ιστός – Η ιστοσελίδα R περιέχει σύνδεσμο προς την ιστοσελίδα Q



Κοινωνικά Δίκτυα - Facebook

Υπάρχει, όμως, ένα ακόμη καλύτερο παράδειγμα που απεικονίζει τις καθημερινές μας αλληλεπιδράσεις και η υλοποίησή του στηρίζεται στη δομή δεδομένων «γράφος»: τα κοινωνικά δίκτυα.

Το Facebook, ένα τεράστιο κοινωνικό δίκτυο, είναι ένας τύπος γράφου. Και αν σκεφτούμε περισσότερο το πώς λειτουργεί πραγματικά, αρχίζουμε να κατανοούμε καλύτερα τον τρόπο διασυνδέσεων των κόμβων καθώς και με ποιον τύπο γράφου αναπαριστάνεται. Στο Facebook, εάν σε προσθέσω σαν φίλο, πρέπει να αποδεχτείς το αίτημά μου. Δεν είναι δυνατόν να είμαι φίλος σου στο δίκτυο χωρίς να είσαι και δικός μου. Η σχέση μεταξύ δύο χρηστών (κόμβοι), είναι αμφίδρομη. Δεν υπάρχει η έννοια κόμβου «προέλευσης» και «προορισμού» - αντ' αυτού, είσαι φίλος μου και είμαι δικός σου. Ποιον τύπο γράφου θα χρησιμοποιήσουμε για την αναπαράσταση του κοινωνικού δικτύου Facebook;



Κοινωνικά Δίκτυα - Twitter

A) Το Twitter, από την άλλη πλευρά, λειτουργεί πολύ διαφορετικά από το Facebook. Μπορώ να σε ακολουθήσω, αλλά δεν απαιτείται να με ακολουθήσεις. Με ποιον τύπο γράφου μπορούμε να αναπαραστήσουμε το Twitter;

Θα μπορούσαμε να αναπαραστήσουμε το Twitter ως κατευθυνόμενο γράφο; Κάθε ακμή που δημιουργούμε αντιπροσωπεύει μια μονόδρομη σχέση. Όταν με ακολουθείς στο Twitter, δημιουργείται μια ακμή στον γράφο με τον λογαριασμό σου ως κόμβο προέλευσης και τον λογαριασμό μου ως τον κόμβο προορισμού;

B) Τι θα συμβεί εάν αποφασίσω να σε ακολουθήσω και εγώ; Αλλάζω την κατεύθυνση της ακμής που δημιουργήθηκε όταν με ακολουθούσες; Ξαφνικά η ακμή γίνεται αμφίδρομη και επομένως μη κατευθυνόμενη;



Επτά γέφυρες του Königsberg

Υπάρχει μια ενδιαφέρουσα ιστορία πίσω από την προέλευσή του προβλήματος «οι Επτά γέφυρες του Königsberg». Η πρόκληση (ή απλά σπαζοκεφαλιά) με τις γέφυρες του Königsberg, ήταν να μπορεί κάποιος να διασχίσει και τις επτά γέφυρες μόνο μία φορά περπατώντας μέσα στην πόλη. Κοιτάξτε πρώτα την Εικόνα 1.3.30 για να αντιληφθείτε το πρόβλημα:



Εικόνα 1.3.30. Οι επτά γέφυρες του Königsberg

Προσπαθήστε και δείτε αν μπορείτε να περπατήσετε μέσα από την πόλη και να διασχίσετε όλες τις γέφυρες μόνο μία φορά. Θα πρέπει να λάβετε υπόψη δύο πράγματα ενώ προσπαθείτε να λύσετε το παραπάνω πρόβλημα-αίνιγμα:

- πρέπει να διασχίσετε όλες τις γέφυρες
- δεν πρέπει να διασχίσετε περισσότερο από μία φορά κάποια γέφυρα

Μπορείτε να δοκιμάσετε οποιονδήποτε αριθμό συνδυασμών θέλετε, αλλά θα δείτε ότι είναι αδύνατον. Δεν υπάρχει κανένας τρόπος με τον οποίο κάποιος μπορεί να περπατήσει μέσα στην πόλη και να διασχίσει κάθε γέφυρα μόνο μία φορά. Ο Leonhard Euler αναπαράστησε αυτό το πρόβλημα σε γράφο, όπου αναπαράστησε τα τμήματα γης (A,B,C,D) με κορυφές και τις γέφυρες με ακμές.

Αναζητήστε στο Διαδίκτυο πως ο Euler αναπαράστησε το πρόβλημα με γράφο και για ποιο λόγο είναι αδύνατο να λυθεί αυτό το πρόβλημα.

1.3.4 Ερωτήσεις - Ασκήσεις

Ε.1: Δώστε δύο παραδείγματα εφαρμογών από την καθημερινή ζωή

- απλά συνδεδεμένης λίστας
- διπλά συνδεδεμένης λίστας

Ε.2: Βρείτε το σωστό.

Χαρακτηρίστε τις παρακάτω προτάσεις ως Σωστές ή Λάθος. Στην περίπτωση που πιστεύετε ότι είναι λανθασμένες δικαιολογήστε την επιλογή σας.

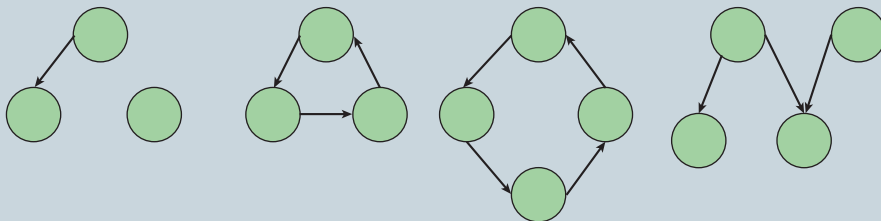
α/α	Προτάσεις	Σ	Λ
1	Μια απλά συνδεδεμένη λίστα μπορούμε να τη διατρέξουμε και προς τις δύο κατευθύνσεις.	☐	☐
2	Σε μία λίστα δε χρειάζεται να οριστεί ένα αρχικό μέγεθος.	☐	☐
3	Δεν είναι δυνατό να υπάρχει «τυχαία» πρόσβαση σε μια απλά συνδεδεμένη λίστα.	☐	☐
4	Σε μια λίστα, τα στοιχεία δεν μπορούν να προστεθούν ή να αφαιρεθούν από τη μέση της λίστας, παρά μόνο από την αρχή ή το τέλος της.	☐	☐
5	Στη διπλά συνδεδεμένη λίστα τα περιεχόμενα των κόμβων προσπελαύνονται και από τις δύο κατευθύνσεις.	☐	☐

Ε.3: Σχεδιάστε γεωγραφικές ιεραρχίες

Σχεδιάστε ένα δένδρο που θα αποτυπώνει την ιεραρχία μιας γεωγραφικής περιοχής σε επίπεδο χωρών, νομών και πόλεων.

Ε.4: Δένδρα ή Γράφοι;

Στην Εικόνα 1.3.31 ποιες από τις παρακάτω δομές είναι δένδρα και ποιες είναι γράφοι. Προσπαθήστε να εξηγήσετε το γιατί.



α. _____ β. _____ γ. _____ δ. _____

Εικόνα 1.3.31. Εντοπίστε τα δένδρα και τους γράφους

E.5: Επιλέξτε τη σωστή απάντηση.

1. Ποια από τις βασικές δομές δεδομένων είναι η πιο κατάλληλη για να αναπαραστήσετε τη δομή των καταλόγων, των υποκαταλόγων και των αρχείων στον σκληρό σας δίσκο:

- ☐ πίνακας
- ☐ λίστα
- ☐ δένδρο
- ☐ ουρά
- ☐ στοίβα

2. Ο κατάλογος των φοιτητών που εγγράφονται σε ένα μάθημα είναι ταξινομημένος αλφαβητικά με βάση το ονοματεπώνυμο και περιλαμβάνει ένα σύνολο πληροφοριών σχετικών με τον φοιτητή, όπως είναι ο κωδικός του φοιτητή, η ημερομηνία γέννησης, το φύλο, η διεύθυνση, ο αριθμός τηλεφώνου κ.λπ. Επιλέξτε ποια από τις παρακάτω δομές δεδομένων είναι καταλληλότερη για την αναπαράσταση αυτών των πληροφοριών:

- ☐ στοίβα
- ☐ δένδρο
- ☐ λίστα
- ☐ ουρά

E.6: Εγκυκλοπαίδεια

Οι πληροφορίες σε μια εγκυκλοπαίδεια μπορούν να οργανωθούν σε ένα σύνολο θεματικών κατηγοριών, όπως Θετικές Επιστήμες, Ιστορία, Τέχνες κ.λπ. Κάθε θεματική κατηγορία μπορεί με την σειρά της να υποδιαιρείται σε μια άλλη σειρά θεματικών υποκατηγοριών σε διάφορα επίπεδα και υποεπίπεδα με τα άρθρα να βρίσκονται στη βάση της δομής. Για παράδειγμα, η κατηγορία Θετικές Επιστήμες μπορεί να περιλαμβάνει τις υποκατηγορίες Μαθηματικά, Φυσική, Χημεία, Πληροφορική κ.λπ.

Επιλέξτε ποια από τις παρακάτω δομές δεδομένων είναι καταλληλότερη για την αναπαράσταση αυτών των πληροφοριών:

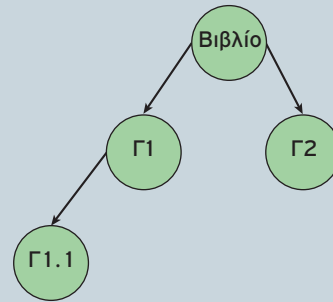
i) δένδρο **ii)** λίστα **iii)** πίνακας

E.7: Περιεχόμενα βιβλίου

Στην Εικόνα 1.3.32 παρουσιάζεται αριστερά η δομή των περιεχομένων ενός βιβλίου και δεξιά ένα ημιτελές δένδρο, το οποίο αναπαριστά εν μέρει την δομή του βιβλίου αυτού. Προσπαθήστε να συμπληρώσετε το δένδρο αυτό, ώστε να απεικονίζει την δομή του βιβλίου στο σύνολό της.

Βιβλίο

Γ1
 Γ1.1
 Γ1.2
 Γ2
 Γ2.1
 Γ2.1.1
 Γ2.1.2
 Γ2.2
 Γ2.3
 Γ3

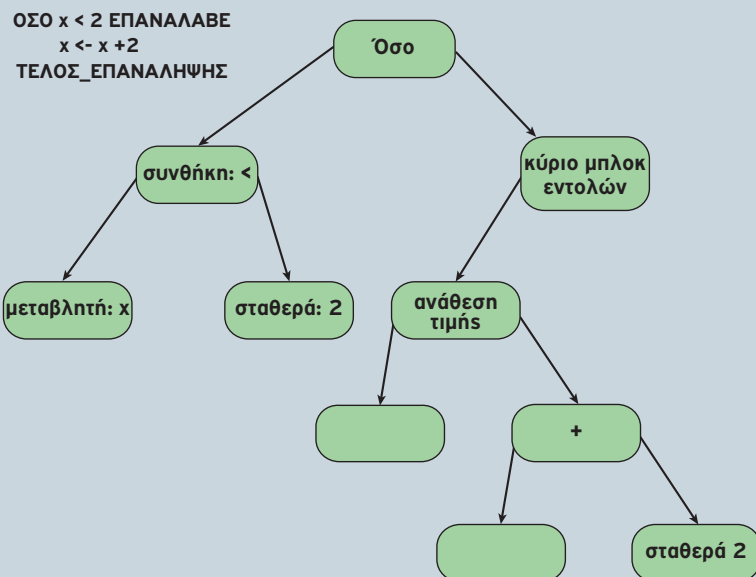


Εικόνα 1.3.32. Περιεχόμενα βιβλίου

Διαγραμματική αναπαράσταση

Ε.8: Δένδρο για την αναπαράσταση κώδικα

Προσπαθήστε να έρθετε στη θέση του μεταγλωττιστή του υπολογιστή σας και να μετατρέψετε το πρόγραμμα της Εικόνας 1.3.33 στο αντίστοιχο δένδρο. Εμείς το κάναμε για εσάς αλλά παραλείψαμε να δώσουμε περιεχόμενο σε κάποιους κόμβους. Μπορείτε να συμπληρώσετε το περιεχόμενο εσείς;



Εικόνα 1.3.33. Δομή τμήματος προγράμματος

Ε.9: Βρείτε το σωστό.

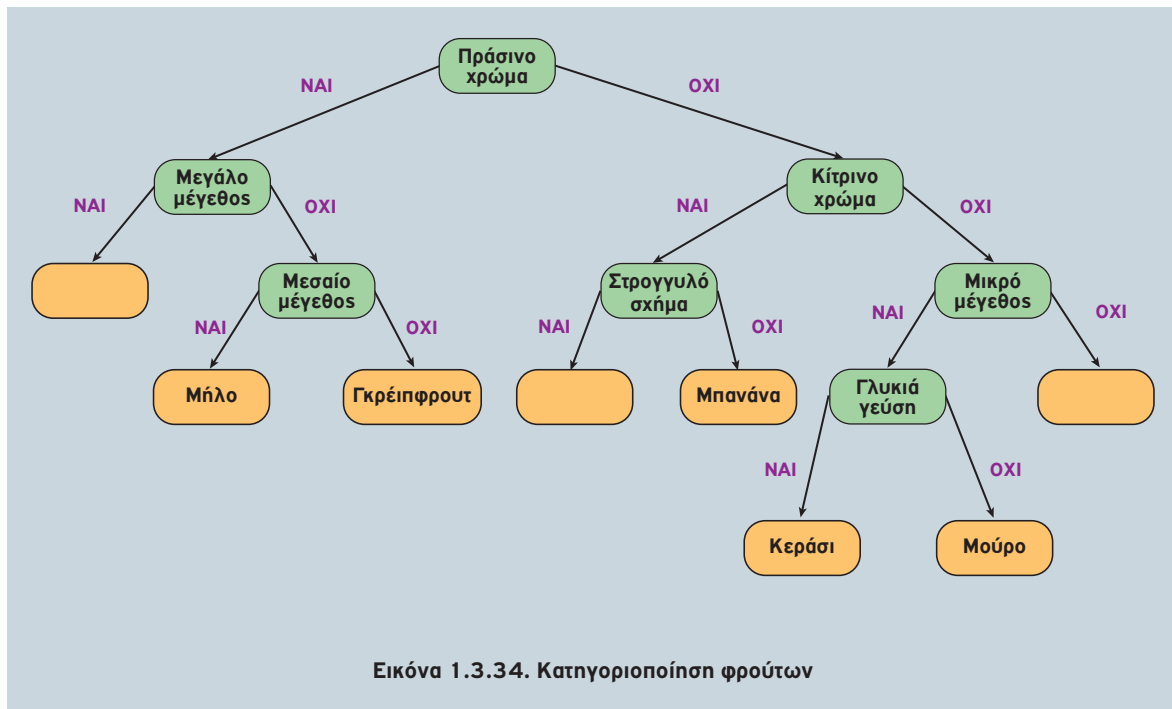
Χαρακτηρίστε τις παρακάτω προτάσεις ως Σωστές ή Λάθος. Στην περίπτωση που πιστεύετε ότι είναι λανθασμένες δικαιολογήστε την επιλογή σας.

α/α	Προτάσεις	Σ	Λ
1	Η ρίζα ενός δένδρου δεν μπορεί ποτέ να είναι φύλλο.	☐	☐
2	Σε ένα δυαδικό δένδρο, φύλλα συναντάμε μόνο στο αριστερό υποδένδρο.	☐	☐
3	Σε ένα δυαδικό δένδρο, κάθε κόμβος-γονέας μπορεί να έχει το πολύ δύο παιδιά	☐	☐
4	Δεν είναι δυνατό να υπάρχουν δύο διαφορετικές διαδρομές από την ρίζα προς έναν άλλον κόμβο ενός δένδρου.	☐	☐
5	Σε ένα δυαδικό δένδρο, κάθε κόμβος έχει μηδέν, ένα ή δύο υποδένδρα.	☐	☐
6	Η ρίζα ενός δένδρου είναι ο μόνος κόμβος ενός δένδρου που δεν έχει γονέα.	☐	☐
7	Τα φύλλα ενός δένδρου είναι απομονωμένοι κόμβοι που δε συνδέονται με άλλους κόμβους.	☐	☐
8	Σε ένα δένδρο, κάθε κόμβος-γονέας μπορεί να έχει οποιονδήποτε αριθμό παιδιών	☐	☐
9	Μπορούν να υπάρχουν διαφορετικές δομές δυαδικών δένδρων αναζήτησης που αποθηκεύουν τα ίδια στοιχεία.	☐	☐
10	Κάθε δένδρο είναι γράφος	☐	☐

Ε.10: Δένδρο απόφασης

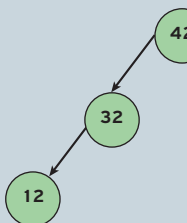
Το δένδρο απόφασης της Εικόνας 1.3.34 αποτελεί μια προσπάθεια κατηγοριοποίησης μιας σειράς φρούτων, όπως το καρπούζι, το μήλο, το γκρέιπφρουτ, το λεμόνι, η μπανάνα, το κεράσι, το μούρο και το πορτοκάλι, με βάση τα χαρακτηριστικά τους, όπως το χρώμα, το μέγεθος, το σχήμα και η γεύση.

Κάποια φρούτα, όμως, όπως το καρπούζι, το πορτοκάλι και το λεμόνι δεν έχουν κατηγοριοποιηθεί ακόμα. Προσπαθήστε να τα ταξινομήσετε ονοματίζοντας τα κενά φύλλα με τις λέξεις καρπούζι, πορτοκάλι και λεμόνι.

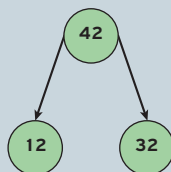


Ε.11: Χαμένοι στο δάσος των δυαδικών δένδρων

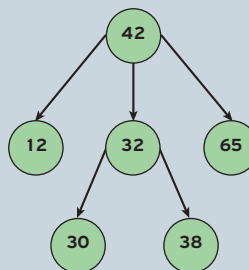
Ποια από τα παρακάτω δένδρα είναι δυαδικά δένδρα αναζήτησης. Εξηγείστε το γιατί:



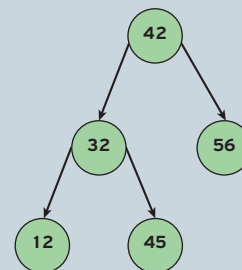
α. _____



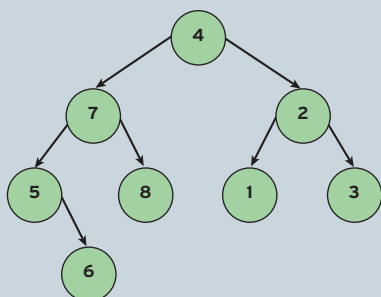
β. _____



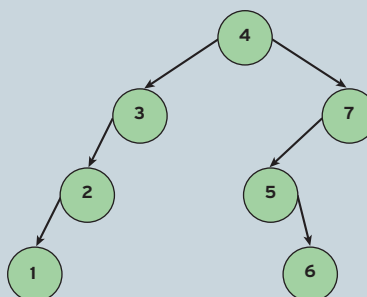
γ. _____



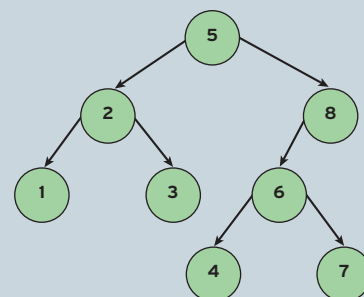
δ. _____



α. _____



β. _____

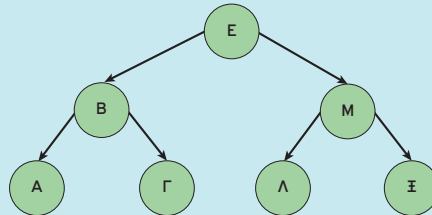


γ. _____

Ε.12: Δεξιά ή αριστερά;

Στο δυαδικό δένδρο αναζήτησης της Εικόνας 1.3.35, αφού συγκρίνετε το στοιχείο Π (το στοιχείο που προσπαθείτε να βρείτε) με το στοιχείο Ε (τη ρίζα) και βρείτε ότι δεν είναι ίσα, σε ποια κατεύθυνση θα το αναζητήσετε στη συνέχεια;

- ☐ Στα αριστερά του Ε
- ☐ Στα δεξιά του Ε
- ☐ Δεν μπορείτε να πείτε και πρέπει να δοκιμάσετε και προς τις δύο κατευθύνσεις.



Εικόνα 1.3.35. Δυαδική αναζήτηση

Ε.13: Τηλεφωνικό Κέντρο Ταξί

Ας υποθέσουμε ότι έχετε τηλεφωνήσει για να έρθει να σας πάρει ένα ταξί. Ένα από τα πιο σημαντικά πράγματα, που είναι κρίσιμα για τη λειτουργία του κέντρου που καλείτε, είναι η ικανότητά του να συνδυάζει τους οδηγούς με τους επιβάτες με αποτελεσματικό τρόπο. Σκεφτείτε ότι υπάρχουν 6 πιθανά ταξί τα οποία μπορούν να έρθουν εγκαίρως να σας παραλάβουν. Πώς θα επιλέξει το κέντρο να σας παραχωρήσει ένα ταξί; Μπορούμε να χρησιμοποιήσουμε γράφο για να δούμε πώς μπορεί να πραγματοποιηθεί η διαδικασία επιλογής του κατάλληλου ταξί. Το πρώτο ταξί (Α) απέχει 12, το δεύτερο (Β) 5, το τρίτο (Γ) 10, το τέταρτο (Δ) 7, το πέμπτο (Ε) 13 και το έκτο (ΣΤ) 20 χιλιόμετρα.

- i) Να σχεδιάσετε τον γράφο που αναπαριστά το πρόβλημα (πάνω στις ακμές γράψτε τα χιλιόμετρα).
- ii) Ποιον τύπο γράφου χρησιμοποιήσατε;
- iii) Ποιο ταξί θα επιλέξει το κέντρο; Θεωρήστε ότι το μόνο κριτήριο για την επιλογή είναι η χιλιομετρική απόσταση.

Ενότητα 2

ΤΕΧΝΙΚΕΣ ΣΧΕΔΙΑΣΗΣ ΑΛΓΟΡΙΘΜΩΝ

Μέθοδος Διαίρει και Βασίλευε

